

A Project Report on
**SMARTDOC IMAGE-BASED RETRIEVAL SYSTEM FOR GOVERNMENT AND
PERSONAL DOCUMENTS**

Submitted in partial fulfillment of the requirement
for the award of the Degree in

Bachelor of Technology
in
Computer Science and Engineering

Submitted By

SHAIK SHA ALI MASTAN AKRAM	22031A0546
VALLURI GEETHA	23035A0509
PAIDI KEERTHIKA	22031A0533
CHUKKA GOWTHAMI	22031A0512
ALAJANGI CHANDU	22031A0503

Under the esteemed guidance of

Mrs. B. JYOTHI
Assistant Professor(c)



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
UNIVERSITY COLLEGE OF ENGINEERING NARASARAOPET
JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY KAKINADA
NARASARAOPET – 522601, PALNADU, ANDHRA PRADESH

2025-2026

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
UNIVERSITY COLLEGE OF ENGINEERING NARASARAOPET
JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY KAKINADA
NARASARAOPET – 522601, PALNADU, ANDHRA PRADESH**



2025-2026

CERTIFICATE

This is certify that the dissertation entitled “**SMARTDOC IMAGE-BASED RETRIEVAL SYSTEM FOR GOVERNMENT AND PERSONAL DOCUMENT**” that is being submitted by **SHAIK SHA ALI MASTAN AKRAM (22031A0546)**, **VALLURI GEETHA (23035A0509)**, **PAIDI KEERTHIKA (22031A0533)**, **CHUKKA GOWTHAMI (22031A0512)**, **ALAJANGI CHANDU (22031A0503)** in partial fulfilment for the award of Bachelor of Technology in Computer Science and Engineering to the University College of Engineering Narasaraopet, Jawaharlal Nehru Technological University Kakinada is a record of bonafide work carried out by them under our guidance and supervision. The results embedded in this dissertation have not been submitted to any other university/institute for the award of any degree/diploma.

PROJECT SUPERVISOR

Mrs. B. JYOTHI
Assistant Professor(c)
Department of CSE
UCEN JNTUK

HEAD OF THE DEPARTMENT

Dr. G. MADHAVI
Associate Professor & HOD(i/c)
Department of CSE
UCEN JNTUK

External Examiner

DECLARATION

We hereby declare that the work described in the project report entitled “**SMARTDOC IMAGE-BASED RETRIEVAL SYSTEM FOR GOVERNMENT AND PERSONAL DOCUMENTS**” which is submitted by us in partial fulfilment of the requirements for the award of the degree Bachelor of Technology in the department of **COMPUTER SCIENCE AND ENGINEERING** to the college, University College of Engineering Narasaraopet, Jawaharlal Nehru Technological University Kakinada, Andhra Pradesh is a record of original and independent research work done by us during the academic year 2025-2026 under the supervision of **Mrs. B. JYOTHI**, Assistant Professor (c), CSE Department. The work is original and has not been submitted for the award of any Degree or Diploma or Associateship or Fellowship or other similar title to this or any other university.

Name of the Student	Roll No	Signature
SHAIK SHA ALI MASTAN AKRAM	22031A0546	
VALLURI GEETHA	23035A0509	
PAIDI KEERTHIKA	22031A0533	
CHUKKA GOWTHAMI	22031A0512	
ALAJANGI CHANDU	22031A0503	

PLACE:

DATE:

ACKNOWLEDGEMENT

It is needed with a great sense of pleasure and immense sense of gratitude that we acknowledge the help of these individuals. We express our sincere thanks and gratitude to many people who helped, supported, encouraged, and challenged us throughout our project and academic program. We take privilege to express our heartfelt gratitude to Project Supervisor **Mrs. B. JYOTHI**, Assistant Professor (c) of CSE Department and to Project Coordinator **Dr. G. MADHAVI**, Associate Professor of CSE Department, Head of the Department(i/c), for their valuable suggestions and constant motivation that greatly helped us in successful completion of the project.

We express our sincere thanks to our beloved **Dr.Y.S. KISHORE BABU**, Vice-Principal(i/c) and **Prof. Ch. SRINIVASA RAO**, Principal, UCEN JNTUK for their support and constant motivation in successful completion of the project. We are thankful to all the faculty members for extending their kind cooperation and assistance. Finally, we are extremely thankful to our parents and friends for their constant help and moral support.

TEAM MEMBERS

SHAIK SHA ALI MASTAN AKRAM	22031A0546
VALLURI GEETHA	23035A0509
PAIDI KEERTHIKA	22031A0533
CHUKKA GOWTHAMI	22031A0512
ALAJANGI CHANDU	22031A0503

TABLE OF CONTENTS

S.NO	CHAPTER NAME	PAGE NO.
	ABSTRACT	i
	LIST OF FIGURES	ii-iii
	LIST OF TABLES	iv
	LIST OF ABBREVIATIONS	v-vi
1.	INTRODUCTION	1
	1.1 INTRODUCTION	2
	1.2 SIGNIFICANCE OF THE PROJECT	2-3
	1.3 SUMMARY	3
2.	LITERATURE SURVEY	4
	2.1 LITERATURE SURVEY	5-8
	2.2 PROBLEM STATEMENT	8
	2.3 SUMMARY	8
3.	SYSTEM ANALYSIS	9
	3.1 EXISTING SYSTEM	10
	3.1.1 DRAWBACKS OF EXISTING SYSTEM	10
	3.2 PROPOSED SYSTEM	11
	3.2.1 ADVANTAGES OF PROPOSED SYSTEM	11
	3.2.2 OVERCOMING THE DRAWBACKS OF THE EXISTING SYSTEM	12
	3.3 DATASET DESCRIPTION	12
	3.4 SUMMARY	12

4.	SOFTWARE REQUIREMENTS SPECIFICATION	13
	4.1 HARDWARE REQUIREMENTS	14
	4.2 SOFTWARE REQUIREMENTS	14
	4.3 FUNCTIONAL REQUIREMENTS	15
	4.4 NON-FUNCTIONAL REQUIREMENTS	15
	4.5 SUMMARY	15
5.	SYSTEM DESIGN	16
	5.1 SYSTEM ARCHITECTURE	17
	5.2 MODULES	18-20
	5.3 TEXT PREPROCESSING	20-21
	5.4 DATA ANALYSIS	21
	5.5 DATA VISUALIZATION	22
	5.6 LANGUAGE MODEL BASED DOCUMENT ANALYSIS	22
	5.7 TEXT CLASSIFICATION AND SEMANTIC ANALYSIS	23
	5.8 LIBRARIES USED IN THE PROJECT	24
	5.9 UML DIAGRAMS	24
	5.9.1 USE CASE DIAGRAM	25-26
	5.9.2 CLASS DIAGRAM	27-28
	5.9.3 SEQUENCE DIAGRAM	29-30
	5.9.4 ACTIVITY DIAGRAM	31-32
	5.9.5 COMPONENT DIAGRAM	33-34
	5.9.6 DATAFLOW DIAGRAM	35-36

	5.10 SUMMARY	37
6.	SYSTEM IMPLEMENTATION	38
	6.1 LIBRARIES	39-42
	6.2 SAMPLE CODE	43-46
	6.3 SUMMARY	47
7.	SYSTEM TESTING	48
	7.1 INTRODUCTION	49
	7.2 PURPOSE OF TESTING	50
	7.3 TYPES OF TESTING	51-54
	7.4 TESTING METHODS	54-58
	7.5 SUMMARY	59
8.	SYSTEM RESULTS	60
	8.1 USER INTERFACE	61-66
	8.2 OCR MODEL PERFORMANCE MATRIX	67
	8.3 SYSTEM PERFORMANCE ANALYSIS	68
	8.4 SUMMARY	69
9.	CONCLUSIONS AND FUTURE WORK	70
	9.1 CONCLUSIONS	71
	9.2 FUTURE WORK	72
	REFERENCES	73-74

ABSTRACT

ABSTRACT

Managing and retrieving scanned personal and government documents such as Aadhaar cards, PAN cards, educational certificates, identity proofs, and invoices has become a major challenge due to unorganized digital storage and inconsistent file naming practices. Users often struggle to locate important documents because traditional retrieval methods rely heavily on remembering file names, folder locations, or manual browsing. These approaches are inefficient, time-consuming, and unsuitable for handling large collections of scanned images. This project presents an intelligent **image-based document retrieval system** that enables users to search for documents by uploading a reference image instead of using conventional text-based queries. The system extracts visual features from the input image and performs similarity matching with stored documents to identify the most relevant results. To enhance understanding, **Optical Character Recognition (OCR)** is employed to extract textual content from retrieved documents. The extracted text is further processed using a **Large Language Model (LLM)** to generate meaningful semantic labels, metadata, and concise document summaries. The novelty of the proposed system lies in the integration of lightweight image retrieval techniques with language-based document understanding, while avoiding computationally expensive deep-learning models. This design ensures faster processing, offline usability, and low resource consumption. The system is evaluated based on retrieval accuracy, response time, and usability, demonstrating effective performance in real-world scenarios. The proposed solution significantly improves document accessibility, organization, and search efficiency, making it highly suitable for personal users, educational institutions, and administrative offices where quick and reliable access to scanned documents is essential. The exponential growth of digital data in organizations, educational institutions, and personal environments has created a significant demand for efficient and intelligent document management systems. Traditional approaches to document handling, which rely on manual filing systems or basic digital storage, are often inadequate for managing large volumes of unstructured and semi-structured data. These systems typically lack advanced capabilities such as intelligent indexing, semantic search, and automated content extraction, resulting in inefficient document retrieval, increased time consumption, and higher chances of data mismanagement. The system demonstrates the effectiveness of integrating OCR, feature extraction, and LLM-based analysis into a unified platform for intelligent document handling.

LIST OF FIGURES

S.NO	FIG NO	DESCRIPTION	PAGE NO
1.	5.1	SYSTEM ARCHITECTURE	17
2.	5.9.1	USE CASE DIAGRAM OF SMARTDOC IMAGE-BASED RETRIEVAL SYSTEM	25
3.	5.9.2	CLASS DIAGRAM OF SMARTDOC IMAGE-BASED RETRIEVAL SYSTEM	27
4.	5.9.3	SEQUENCE DIAGRAM OF SMARTDOC IMAGE-BASED RETRIEVAL SYSTEM	29
5.	5.9.4	ACTIVITY DIAGRAM OF SMARTDOC IMAGE-BASED RETRIEVAL SYSTEM	31
6.	5.9.5	COMPONENT DIAGRAM OF SMARTDOC IMAGE-BASED RETRIEVAL SYSTEM	33
7.	5.9.6	DATA FLOW DIAGRAM	35
8.	7.3.1	UNIT AND JUNIT TESTING OF SMART DOC PROJECT	51
9.	7.3.2	SIMILAR DOCUMENTS RETRIEVED BY THE SMARTDOC SEARCH ENGINE.	52
10.	7.3.3	SIMILAR DOCUMENTS RETRIEVED BY THE SMARTDOC SEARCH ENGINE.	53
11.	7.4.1	BLACK BOX TESTING	54
12.	7.4.2	WHITE BOX TESTING	55
13.	7.4.3	ORGANIZATION OF SMARTDOC TEST MODULES.	56
14.	7.4.4	SIMILARITY SCORES PRODUCED DURING DOCUMENT COMPARISON.	58
15.	8.1.1	SMARTDOC MAIN INTERFACE – DOCUMENT SEARCH PANEL	62

16.	8.1.2	SMARTDOC SEARCH RESULTS PANEL	64
17.	8.1.3	INDEXING PROGRESS AND SYSTEM STATUS MESSAGES	65
18.	8.1.4	DOCUMENT VIEW OPTION AND RESULT VERIFICATION	65

LIST OF TABLES

S.NO	TABLE NO	DESCRIPTION	PAGE NO
1.	7.3.3.1	AVERAGE PERFORMANCE RESULTS	54
2.	7.4.2.1	TESTING TOOLS	55
3.	7.4.3.1	FUNCTIONALITY OF SMARTDOC MODULES	57
4.	7.4.3.2	FUNCTIONAL TEST CASE MATRIX	58

LIST OF ABBREVIATIONS

S.NO	ABBREVIATION	EXPANSION
1.	OCR	Optical Character Recognition
2.	ORB	Oriented FAST and Rotated BRIEF
3.	OpenCV	Open Source Computer Vision Library
4.	AHash	Average Hashing
5.	SHA-256	Secure Hash Algorithm 256-bit
6.	GUI	Graphical User Interface
7.	UI	User Interface
8.	DB	Database
9.	JSON	JavaScript Object Notation
10.	API	Application Programming Interface
11.	JVM	Java Virtual Machine
12.	IDE	Integrated Development Environment
13.	RAM	Random Access Memory
14.	CPU	Central Processing Unit
15.	PNG	Portable Network Graphics
16.	JPG	Joint Photographic Experts Group
17.	JPEG	Joint Photographic Experts Group
18.	PDF	Portable Document Format
19.	XML	Extensible Markup Language
20.	HTTP	HyperText Transfer Protocol
21.	JavaFX	Java-based Graphical User Interface Framework
22.	Tesseract OCR	Open Source Optical Character Recognition Engine

23.	ML	Machine Learning
24.	DFD	Data Flow Diagram
25.	UML	Unified Modeling Language
26.	CSV	Comma-Separated Values
27.	OS	Operating System
28.	GPU	Graphics Processing Unit
29.	SSD	Solid State Drive

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION

The rapid growth of digital documentation has led individuals and organizations to store large numbers of scanned files such as Aadhaar cards, PAN cards, educational certificates, invoices, and identity proofs. Although these documents are preserved in digital form, they are often scattered across devices and folders with inconsistent or meaningless file names, making retrieval difficult and time-consuming. Traditional keyword-based search methods are ineffective when users do not remember the exact file name or textual content inside the document.

This project proposes an **image-based document retrieval system** that allows users to search for documents by simply uploading a reference image. Instead of relying solely on filenames or manual tagging, the system analyzes the visual characteristics of the input image and compares them with stored documents to identify the most similar matches. To enhance understanding, **Optical Character Recognition (OCR)** is used to extract embedded text, and a **Large Language Model (LLM)** processes this text to generate meaningful labels and summaries.

The system is designed to be lightweight and efficient by avoiding heavy deep-learning architectures while still providing accurate and fast retrieval. It aims to simplify document management for personal, institutional, and government environments where quick access to identity and record files is essential.

1.2 SIGNIFICANCE OF THE PROJECT

Efficient document retrieval has become a critical requirement in the digital era. The proposed system offers several important benefits:

1. **User-friendly Search Mechanism:** Users can find documents using an image instead of remembering file names, dates, or locations.
2. **Improved Organization:** Automatically generated semantic labels and summaries help categorize documents in a structured manner.
3. **Time and Effort Reduction:** Eliminates manual browsing through folders and speeds up access to important records.
4. **Practical for Real-World Use:** Suitable for students, employees, government offices, and small organizations handling large volumes of scanned files.
5. **Lightweight and Cost-effective:** The system avoids computationally expensive deep learning models, making it deployable on low-resource devices.

6. **Enhanced Security and Accessibility:** Quick retrieval of identity proofs and certificates can assist in verification processes and emergency situations.

By integrating image similarity, OCR, and language understanding, the project bridges the gap between visual search and intelligent document management.

1.3 SUMMARY

The proposed project presents a smart and efficient solution for retrieving scanned personal and government documents using an image-based approach. Users can upload a sample image, and the system identifies matching documents through visual feature comparison. Extracted text from OCR is further analyzed by a language model to produce semantic descriptions, improving search relevance and organization.

The system focuses on achieving high retrieval accuracy with minimal computational overhead, making it practical for everyday use. Evaluation based on accuracy and response time demonstrates its effectiveness compared to conventional text-based search methods. Overall, the project enhances accessibility, organization, and usability of digital documents and can be extended for institutional record management in the future.

The rapid growth of digital documents has created a need for efficient and intelligent document management systems. Traditional methods of storing and retrieving documents are often manual or rely on basic file systems, making organization and search time-consuming and inefficient. To address these challenges, SmartDoc is developed as a fully local, offline document management and search system.

SmartDoc integrates technologies such as OCR for text extraction, feature extraction for image analysis, and local language models for document understanding. It enables users to upload, organize, index, and search documents using both textual and visual similarities. With a user-friendly interface and secure local storage, the system ensures data privacy while improving document accessibility and retrieval efficiency.

CHAPTER 2

LITERATURE SURVEY

2.1 LITERATURE SURVEY

The increasing growth of digital data has created a strong need for intelligent document management systems. Traditional storage methods lack advanced indexing, semantic search, and automated content extraction, leading to inefficient retrieval and poor organization. Research highlights the importance of structured storage, metadata management, and secure access control. Technologies such as OCR enable text extraction from scanned documents, while NLP and transformer-based models support semantic search, summarization, and classification. Feature extraction techniques like ORB enhance image-based retrieval, and lightweight databases such as SQLite provide reliable local storage. Real-time monitoring further ensures automatic indexing and updated records. By integrating these technologies, SmartDoc offers a secure, offline, and intelligent document management solution that improves efficiency and data handling. Overall, the literature highlights the importance of integrating multiple techniques—OCR, feature extraction, semantic analysis, and efficient database systems—to build intelligent document management solutions.

[1]F. Chen and D. Bloomberg. Extraction of thematically relevant text from images. In SDAIR, pages 163-178, 1996

The paper is an early contribution to document image analysis that focuses on extracting meaningful content from scanned documents without fully relying on OCR. It moves beyond basic image processing toward semantic understanding by identifying important text segments and enabling image-based summarization. The study demonstrates that useful thematic information can be obtained even without complete character recognition. By combining computer vision techniques with statistical text analysis, it supports effective document indexing and forms a foundation for modern digital library and document management systems.

[2]V.N. Gudivada and V.V. Raghavan. Spatial similarity based retrieval in image databases. In SDAIR, pages 255-270,1993

Gudivada and Raghavan introduced an early approach to content-based image retrieval that focuses on spatial similarity instead of keywords or simple visual features. Their method converts images into symbolic representations by identifying and encoding objects based on properties like centroids and labels. By comparing spatial relationships and supporting geometric transformations such as rotation, translation, and scaling, the

approach enables more flexible and meaningful image retrieval. This work laid the foundation for spatial-based and semantic image search in large image databases.

[3]J. Hull, J. Cullen, and M. Peairs. Document image matching and retrieval techniques. In SDIUT, pages 31-38, 1997.

The paper presents a method for retrieving document images based on visual similarity instead of text content. It uses a two-stage framework: first identifying candidate matches through feature-based similarity detection, and then confirming exact matches through detailed equivalence detection. The approach includes techniques using symbolic text-region features and features from fax-compressed images. These methods enable applications such as automatic filing, duplicate detection, and document stream monitoring.

[4]P. Herrmann and G. Schlagetar. Retrieval of document images using layout knowledge. In Proc,of the ICDAR, pages 537-540, 1993.

The paper introduces a document image retrieval method based on layout structure rather than textual content or OCR results. It analyzes the spatial arrangement of elements such as text blocks, images, headings, and columns, extracting features like size, position, and relative relationships between components. These layout features are stored in a database and used for comparison. A key contribution is the layout-based query mechanism, where users can search for documents by specifying a desired page structure. This approach is especially useful when text recognition is unreliable or not required, enabling effective retrieval through structural similarity.

[5]K. Tsuda, S. Senda, M. Minoh, and K. Ikeda. Clustering ocr-ecl texts for browsing document image database. In Proc. of the ICDAR, pages 171-174, 1995.

The paper focuses on improving browsing in large document image databases where OCR results may be noisy or inaccurate. It proposes clustering OCR-extracted text based on overall textual similarity, allowing related documents to be grouped together. This approach reduces the impact of OCR errors and enables topic-oriented browsing, making it easier for users to navigate and explore scanned document collections efficiently.

[6]F. R. Chen, L. D. Wilcox, and D. S. Bloomberg. Detecting and locating partially specified keywords in scanned images using hidden Markov models. In Proc.of the ICDAR, pages 133 - 138, 1993.

Chen, Wilcox, and Bloomberg (1993) propose a method for detecting partially specified keywords in scanned document images using Hidden Markov Models (HMMs). Instead

of depending entirely on OCR output, their approach uses statistical modeling to identify keyword patterns directly from image data. The method is robust to noise and recognition errors, making it effective for keyword spotting and document retrieval when OCR results are incomplete or unreliable.

[7]Gerard Salton and Michael J. McGill. Introduction to Modern Information Retrieval. McGraw-Hill, New York, 1983.

Salton and McGill's *Introduction to Modern Information Retrieval* (1983) is a foundational text in the field of information retrieval (IR). The book introduces core concepts such as indexing, document representation, vector space models, term weighting (including TF-IDF), and similarity measurement techniques. It provides both theoretical foundations and practical approaches for designing systems that retrieve relevant documents from large collections of data. The work significantly influenced the development of search engines, digital libraries, and database retrieval systems, establishing key principles that are still widely used in modern information retrieval and search technologies.

[8]R.K. Srihari. Automatic indexing and content-based retrieval of captioned photographs. In Proc. of the ICDAR, pages 1165-1168, 1995.

R.K. Srihari (1995) presents a method for automatic indexing and content-based retrieval of captioned photographs. The paper explores how both visual features of images and accompanying textual captions can be integrated to improve retrieval performance. By combining image analysis techniques with natural language processing of captions, the system enables more accurate indexing and searching of photographic databases. This work highlights the importance of multimodal retrieval—using both visual and textual information—and contributes to the early development of intelligent image retrieval systems in digital libraries and document archives.

[9]D. Doermann and C. Shin. The development of a general framework for intelligent document image retrieval. In DAS, pages 605-632, 1996

Doermann and Shin (1996) present a general framework for intelligent document image retrieval that integrates image analysis, feature extraction, indexing, and retrieval processes. Their modular architecture combines layout, textual, and visual features to improve search accuracy in large document collections. The work provides a foundational design for scalable and extensible document image retrieval systems.

[10]M. Amlani and R. Kasturi. A query processor for information extraction from images of paper-based maps. In RIAO 88, pages 991-1000, 1988.

Amlani and Kasturi (1988) present a query processing system designed for extracting information from images of paper-based maps. The paper focuses on analyzing map images to identify and retrieve specific geographic or symbolic information based on user queries. The system processes visual elements such as symbols, text labels, and spatial relationships to interpret and respond to structured queries. This work represents an early effort in image-based information extraction, demonstrating how computer vision techniques can be applied to complex document images like maps. It laid important groundwork for spatial query processing and content-based retrieval in graphical and document image databases.

2.2 PROBLEM STATEMENT

Traditional document management systems in organizations and institutions rely on manual filing or basic digital storage, which lack intelligent search and efficient organization. As document volumes grow, filename-based search and manual categorization become time-consuming and ineffective. Scanned documents further complicate retrieval due to the need for OCR, which may produce errors and reduce accuracy. The absence of visual similarity search, semantic understanding, and secure centralized storage increases the risk of duplication and mismanagement. SmartDoc addresses these challenges by providing a secure, fully local system that integrates OCR, feature extraction, local language models, and database indexing to enable intelligent, efficient, and privacy-focused document management.

2.3 SUMMARY

Existing document management systems rely on manual organization and basic file storage, making document retrieval slow and inefficient as data volumes increase. Scanned documents are difficult to search without OCR, and recognition errors further reduce accuracy. There is a need for a secure, intelligent, and fully offline system that supports automated content extraction, efficient indexing, visual and semantic search, and structured storage. SmartDoc addresses these challenges by integrating OCR, feature extraction, local language models, and database management into a centralized and privacy-focused document management platform.

CHAPTER 3

SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

In the present method of document management, individuals and organizations store scanned copies of important documents such as Aadhaar cards, PAN cards, mark sheets, certificates, invoices, and identity proofs in local computers, mobile devices, or cloud storage platforms. These documents are usually saved as image or PDF files without following any standard naming convention or structured categorization.

Retrieval of documents in the existing system is mainly performed through manual browsing of folders or by using simple text-based search options provided by operating systems. This approach assumes that the user remembers the exact file name, date of creation, or storage location. However, in real-world scenarios, users rarely remember such details, especially when the number of documents is large.

Moreover, scanned documents are stored as images, and most systems do not extract or analyze the textual content present inside them. As a result, even if a document contains important information, it cannot be searched based on its actual content. The absence of intelligent processing makes the existing system inefficient, unreliable, and highly dependent on human memory.

3.1.1 DRAWBACKS OF EXISTING SYSTEM

The current method of managing and retrieving scanned documents suffers from several limitations:

1. Document retrieval is completely dependent on file names, which are often random or meaningless.
2. There is no facility to search documents using an image as input.
3. Users must manually open multiple files to identify the correct document.
4. Scanned images are not processed to extract embedded textual information.
5. Lack of automatic classification leads to unorganized storage.
6. The process becomes extremely slow when the number of documents increases.
7. No semantic understanding of document content is available.
8. Existing systems do not provide summaries or labels for quick identification.

These drawbacks make the traditional approach unsuitable for managing large collections of personal and government documents.

3.2 PROPOSED SYSTEM

The proposed system introduces an intelligent and automated approach for document retrieval using image-based search. Instead of depending on file names or manual browsing, the user can upload a reference image of a document. The system extracts visual features from the uploaded image and compares them with the features of stored documents to identify the most similar matches.

Once relevant documents are identified, Optical Character Recognition is applied to extract the textual content present in those documents. This extracted text is further processed using a Large Language Model to generate meaningful semantic labels, keywords, and short summaries. These additional details help users to quickly understand the nature of the retrieved document without opening it manually.

The system is designed with efficiency in mind and avoids the use of heavy deep-learning models that require high computational power. Lightweight image processing and similarity techniques are used so that the system can work smoothly on normal computers and mobile devices.

3.2.1 ADVANTAGES OF PROPOSED SYSTEM

The proposed system provides multiple benefits over the traditional method:

1. Users can retrieve documents by uploading an image without remembering file names.
2. Visual similarity matching enables accurate identification of documents.
3. OCR extracts hidden textual information from scanned images.
4. Language model processing adds semantic understanding to documents.
5. Automatic labeling improves organization and management.
6. Retrieval time is significantly reduced.
7. The system is easy to use for non-technical users.
8. It can be applied in homes, colleges, offices, and government institutions.
9. The lightweight design makes it cost-effective and practical.

These advantages make the proposed system a reliable solution for modern document management needs.

3.2.2 OVERCOMING DRAWBACKS OF THE EXISTING SYSTEM

The proposed system directly addresses the problems identified in the existing approach. Dependence on file names is removed by introducing image-based retrieval, where the actual visual appearance of the document is used for searching. The inability to understand document content is solved through OCR and language model integration, which convert image text into meaningful information.

Manual searching is replaced with automated similarity matching, which provides faster and more accurate results. Unorganized storage is improved through semantic labeling and generated summaries, allowing users to identify documents easily. The proposed system therefore transforms a passive collection of files into an intelligent and searchable document repository.

3.3 DATASET DESCRIPTION

The dataset used in this project consists of scanned images of personal and government-related documents such as Aadhaar cards, PAN cards, certificates, identity proofs, and invoices. The documents are stored in image formats like JPG and PNG with different resolutions and quality levels to represent real-world conditions.

The dataset includes both camera-captured and scanned documents containing printed text, logos, stamps, and signatures. No fixed naming convention is followed, simulating unorganized user storage. OCR is applied to extract textual content, and visual features are generated for image similarity matching.

This dataset is used to evaluate document retrieval accuracy, OCR performance, and response time of the proposed system.

3.4 SUMMARY

Traditional document management techniques are inefficient for handling large numbers of scanned files, leading to issues such as unorganized storage and lack of image-based search. The proposed system solves these problems by combining image processing, OCR, and language-based intelligence. It enables quick document retrieval using reference images and improves organization while reducing manual effort.

CHAPTER 4

SOFTWARE REQUIREMENTS SEPCIFICATIONS

SOFTWARE REQUIREMENTS SPECIFICATION

The Software Requirements Specification defines the hardware, software, and functional needs of the proposed image-based document retrieval system. It describes what the system must perform and the conditions under which it should operate.

4.1 HARDWARE REQUIREMENTS

The minimum hardware configuration required to run the system is as follows:

1. Processor: Intel i3 or above
2. RAM: 4 GB minimum (8 GB recommended)
3. Hard Disk: 20 GB free space
4. Camera/Scanner: For capturing document images
5. Display: Standard monitor
6. Input Devices: Keyboard and mouse

4.2 SOFTWARE REQUIREMENTS

1. Operating System: Windows / Linux / macOS
2. Programming Language: Java
3. User Interface: JavaFX
4. Image Processing: OpenCV (ORB Feature Extraction)
5. OCR Engine: Tesseract OCR
6. Language Processing: Local LLM for document analysis
7. Database: SQLite
8. Development Tools: IntelliJ IDEA / Eclipse / VS Code
9. Build Tool: Maven/Gradle
10. External Libraries:
 - OpenCV Java bindings
 - Tess4J (Tesseract wrapper)
 - SQLite JDBC
 - Apache PDFBox (for PDF handling)

4.3 FUNCTIONAL REQUIREMENTS

The system shall perform the following functions:

1. Scan selected folders and discover new documents automatically
2. Extract visual features from documents using ORB algorithm
3. Perform OCR text extraction using Tesseract
4. Analyze extracted text using Local LLM to generate:
 - Document labels
 - Short summaries
 - Metadata insights
5. Store features, text, and metadata in SQLite database
6. Accept a query image from the user through JavaFX interface
7. Perform similarity matching and retrieve top relevant documents
8. Display search results with file location and metadata
9. Monitor folders in real time and trigger re-indexing on changes

4.4 NON-FUNCTIONAL REQUIREMENTS

The system must satisfy the following quality requirements:

1. **Performance:** Document retrieval should be completed within a few seconds
2. **Accuracy:** Similarity matching must return relevant results
3. **Usability:** JavaFX interface should be simple and interactive
4. **Reliability:** System should work offline without internet
5. **Scalability:** Should support increasing number of documents
6. **Security:** Local storage of documents and metadata
7. **Maintainability:** Modular architecture for easy upgrades
8. **Portability:** Should run on multiple operating systems

4.5 SUMMARY

The Software Requirements Specification defines the technical environment and operational needs of the SmartDoc system as per the designed architecture. The system is implemented using Java with JavaFX interface, SQLite database, ORB feature extraction, Tesseract OCR, and Local LLM processing. These requirements ensure that the system performs efficient offline document discovery, intelligent analysis, and fast image-based retrieval in a reliable and user-friendly manner.

CHAPTER 5

SYSTEM DESIGN

SYSTEM DESIGN

System design is an essential step in the software development life cycle, which involves designing the system architecture, modules, components, and interfaces to satisfy the specified requirements. The system design in a project is critical to ensure that the system meets the specified requirements, is scalable, and is maintainable in the long run.

5.1 SYSTEM ARCHITECTURE

A system architecture is the conceptual model that defines the structure, behaviour, and more views of a system. It shows the connections between the various components of the system and indicates what functions are to be performed. SmartDoc follows a **modular layered architecture** that ensures clear separation of responsibilities, scalability and efficient processing of documents. The system is designed to operate completely offline, integrating multiple components such as document scanning, OCR, feature extraction, natural language processing, and database management.

The below figure shows system architecture of our project. This depicts the model building process.

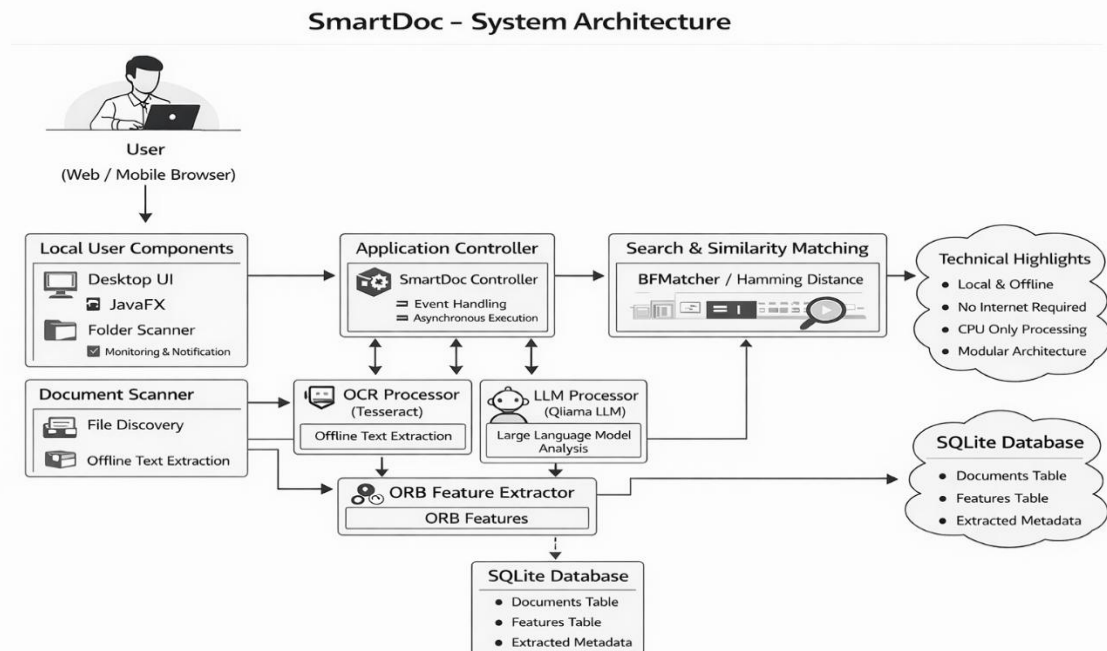


Figure 5.1 System Architecture

5.2 MODULES

5.2.1 Document Collection

Module Functionality

This module is responsible for discovering and collecting scanned documents from user-specified folders. It continuously scans directories to identify files such as Aadhaar cards, PAN cards, certificates, invoices, and identity proofs in formats like JPG, PNG, and PDF. The module also detects newly added or modified files for further processing.

Importance

It serves as the entry point of the system by gathering all available documents and ensuring that no file is missed. This enables automatic indexing without requiring manual uploads.

5.2.2 Image Feature Extraction

Module Functionality

This module extracts visual features from each document using the ORB (Oriented FAST and Rotated BRIEF) algorithm through OpenCV. The extracted features act as a unique fingerprint for every document and are stored in the SQLite database for similarity comparison.

Importance

It enables fast and accurate image-based retrieval by representing documents through their visual characteristics rather than file names.

5.2.3 OCR Text Extraction

Module Functionality

Tesseract OCR is applied to extract embedded textual content from scanned images and PDFs. The module performs preprocessing such as noise removal, resizing, and orientation correction before text extraction.

Importance

Converts non-searchable image documents into machine-readable text, enabling further semantic analysis and metadata generation.

5.2.4 LLM Document Analysis

Module Functionality

The extracted text is processed using a Local Large Language Model to generate:

1. Document type identification
2. Semantic labels

3. Short summaries
4. Important keywords and metadata

Importance

Adds language-based understanding to visual retrieval, allowing users to recognize documents quickly without opening them.

5.2.5 Database Management**Module Functionality**

This module stores and manages:

1. ORB feature vectors
2. OCR extracted text
3. Generated metadata and summaries
4. Document file locations using SQLite database tables.

Importance

Ensures structured storage and fast retrieval of document information during search operations.

5.2.6 Search & Similarity Matching**Module Functionality**

When a user uploads a query image through JavaFX interface, this module extracts its features and compares them with stored features using similarity matching algorithms. Top matching documents are retrieved along with metadata.

Importance

Provides the core image-based search capability of the system with quick and relevant results.

5.2.7 Real-Time Monitoring**Module Functionality**

Monitors selected folders for file changes such as:

1. New document addition
2. Modification of existing files
3. Deletion events and triggers automatic re-indexing when required.

Importance

Maintains up-to-date document index without manual intervention.

5.2.8 User Interface

Module Functionality

A JavaFX-based graphical interface allows users to:

1. Upload query images
2. View retrieved documents
3. See summaries and metadata
4. Navigate to actual file locations

Importance

Provides an interactive and user-friendly environment for both technical and non-technical users.

5.2.9 Result & Reporting

Module Functionality

Displays top matches with:

1. Similarity score
2. Document preview
3. Extracted summary
4. File path information

Importance

Helps users quickly identify the correct document and improves decision making.

5.3 TEXT PREPROCESSING

Text preprocessing is performed to clean and prepare OCR-extracted text before semantic analysis.

The following steps are carried out:

1. Noise Removal

Removal of special characters, unwanted symbols, irregular spacing, and OCR artifacts.

2. Lowercase Conversion

Conversion of all extracted text into lowercase format for uniformity.

3. Tokenization

Splitting text into individual words or meaningful tokens.

4. Stop Word Removal

Elimination of common words such as “the”, “is”, “and”, etc., which do not add meaningful value.

5. Text Normalization

Correction of minor OCR errors and formatting inconsistencies.

6. Clean Text Output

The final structured text is forwarded to the language model for classification and summarization.

5.4 DATA ANALYSIS

The analysis includes:

1. Document Type Identification

Detecting document categories such as Aadhaar card, PAN card, certificate, invoice, etc.

2. Keyword Frequency Analysis

Identifying frequently occurring words for better classification.

3. Visual Feature Analysis

Evaluating ORB descriptors for similarity consistency.

4. Similarity Score Evaluation

Analyzing how accurately matching documents are ranked.

5. Performance Metrics

1. Retrieval Accuracy
2. OCR Extraction Quality
3. Response Time
4. Matching Reliability

6. Indexing

1. Creates searchable indexes for fast retrieval.
2. Supports both text-based and feature-based indexing

7. Error Handling

1. Handles OCR inaccuracies for low-quality images
2. Skips unsupported or corrupted files

5.5 DATA VISUALIZATION

Data visualization provides graphical insights into system performance and document organization.

The system may generate:

1. Document Category Distribution Chart

Displays the number of stored documents.

2. Similarity Score Graph

Shows ranking of retrieved documents based on matching score.

3. Keyword Frequency Chart

Highlights most common terms in stored documents.

4. Performance Visualization

Displays response time and retrieval efficiency.

Visualization helps in monitoring system accuracy and usability.

5.6 LANGUAGE MODEL BASED DOCUMENT ANALYSIS

A Large Language Model (LLM) is used to add semantic understanding to extracted document text.

Key functions include:

1. Context Understanding

Interprets relationships between words in a document.

2. Document Type Detection

Automatically identifies document category based on content.

3. Keyword Extraction

Identifies important entities and identifiers.

4. Automatic Summary Generation

Produces short and meaningful summaries.

5. Semantic Label Creation

Generates structured metadata for better organization.

6. Output Storage

The analyzed results are stored in the database for future search and retrieval.

This module enhances intelligent document recognition beyond visual matching.

5.7 TEXT CLASSIFICATION AND SEMANTIC ANALYSIS

Text classification categorizes documents based on extracted textual content.

The classification process includes:

1. Feature Identification

Detecting important keywords and contextual phrases.

2. Document Categorization

Classifying documents into:

1. Identity Proof
2. Educational Certificate
3. Financial Document
4. Government Record

3. Semantic Interpretation

Understanding document meaning rather than simple keyword matching.

4. Evaluation Metrics

1. Accuracy
2. Precision
3. Recall
4. F1 Score

5. Working Process

1. OCR extracts raw text from document
2. Text is cleaned and preprocessed
3. Input is sent to local LLM (via Ollama)
4. LLM performs:
 - Classification
 - Summarization
 - Keyword extraction
5. Results are stored in the database

This ensures automatic and reliable document organization.

5.8 LIBRARIES USED IN THE PROJECT

The SmartDoc system uses the following tools and libraries:

1. OpenCV

Used for ORB feature extraction and image similarity matching.

2. Tesseract OCR (Tess4J)

Used for extracting text from scanned images and PDFs.

3. SQLite JDBC

Used for storing document features and metadata.

4. JavaFX

Used for developing the graphical user interface.

5. Local Large Language Model

Used for semantic analysis, classification, and summary generation.

6. Apache PDFBox

Used for handling and processing PDF documents.

These libraries ensure efficient, lightweight, and intelligent document retrieval.

5.9 UML DIAGRAMS USED IN DESIGN

UML is a standardized modelling language consisting of an integrated set of diagrams, developed to help system and software developers for specifying, visualizing, constructing, and documenting the artifacts of software systems, as well as for business modelling and other non- software systems. UML is used for describing the system architecture in detail using the blue- print. The UML is a very important part of developing object-oriented software and the soft- ware development process. The UML uses mostly graphical notations to express the design of software projects.

In the SmartDoc project, UML helps in visualizing the overall system design, including user interactions, data flow, and communication between different modules such as OCR processing, feature extraction, LLM analysis, and database management. By using UML diagrams, the complexity of the system is simplified, making it easier to understand, develop, and maintain. Overall, UML plays a crucial role in improving system planning, communication, and documentation, ensuring that all components of the system are well-structured and function as intended.

5.9.1 USE CASE DIAGRAM

A Use Case Diagram is a type of UML (Unified Modeling Language) diagram that shows how users (actors) interact with a system to achieve specific goals. It focuses on system functionality from the user's perspective rather than internal design.

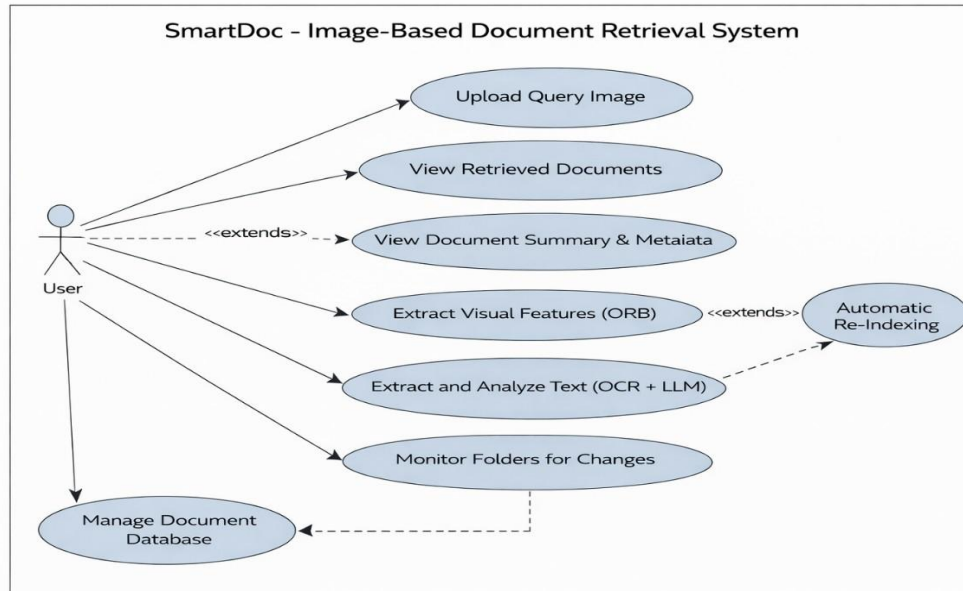


Figure 5.9.1 Use Case Diagram of SmartDoc Image-Based Retrieval system

Actors

1. User

The user interacts with the system through the JavaFX interface to upload a query image, search for documents, and view retrieved results along with summaries and metadata.

2. System (SmartDoc Backend)

The system performs internal operations such as feature extraction, OCR processing, similarity matching, and document indexing without direct user intervention.

Use Cases

1. Upload Query Image

The user uploads a reference image of a document through the JavaFX interface to initiate the search process.

2. View Retrieved Documents

The system displays the list of top matching documents along with file names and storage locations based on similarity results.

3. View Document Summary & Metadata The user views the automatically generated summary, semantic labels, and metadata extracted from the document using OCR and LLM analysis.

4. Extract Visual Features (ORB)

The system extracts ORB visual features from stored documents and from the uploaded query image for comparison.

5. Extract and Analyze Text (OCR + LLM)

The system performs OCR to extract embedded text from documents and analyzes the text using a Local LLM to generate meaningful insights.

6. Similarity Matching

The system compares query image features with stored features to identify the most relevant documents.

7. Monitor Folders for Changes

The system continuously monitors selected directories to detect newly added or modified documents.

8. Automatic Re-Indexing

When changes are detected, the system automatically re-processes the documents and updates the database.

9. Manage Document Database

The system stores and manages ORB features, OCR text, metadata, and file paths in the SQLite database for efficient retrieval.

10. Navigate to File Location

The user can open the actual file location of a retrieved document directly from the interface.

The user interacts with the SmartDoc system to upload or scan documents, which are then processed through OCR, feature extraction, and LLM analysis. The processed data is stored in the database. The user can search for documents using text or images, and the system retrieves and displays relevant results. Additionally, the system supports real-time monitoring of folders for automatic updates.

5.9.2 CLASS DIAGRAM

A Class Diagram is a type of UML (Unified Modeling Language) diagram that shows the structure of a system by representing its classes, their attributes (data), methods (functions), and the relationships between those classes. It provides a static view of how the system is organized and how different components interact with each other.

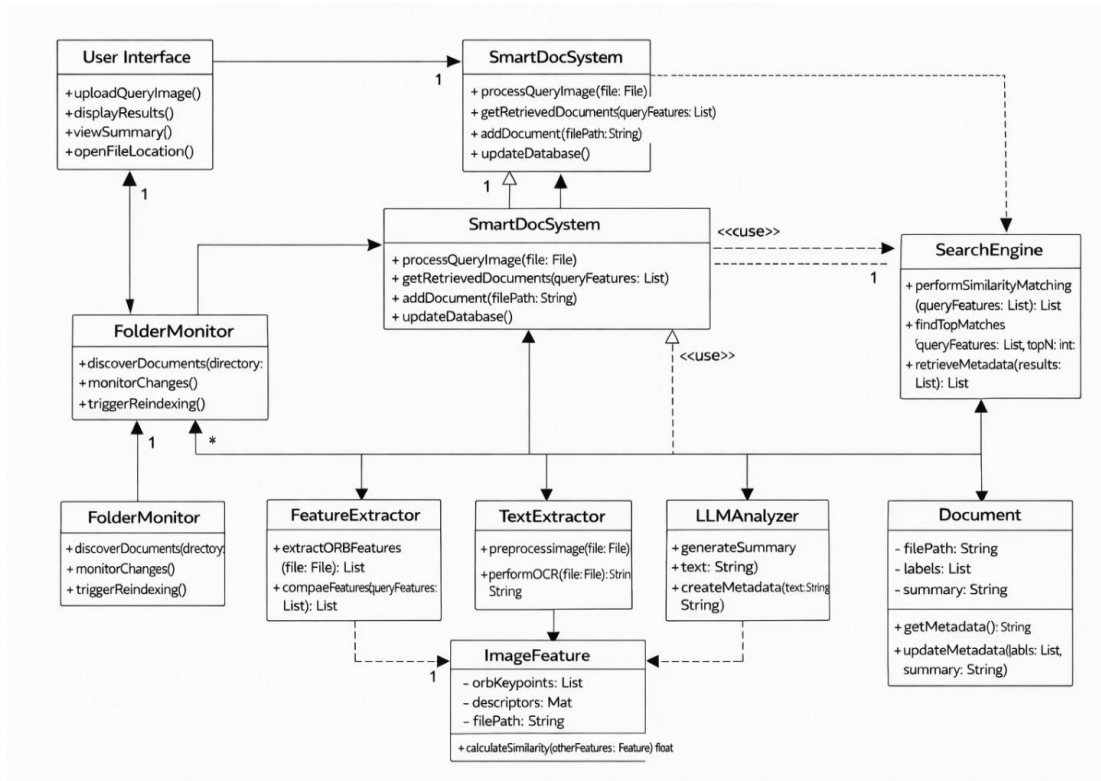


Figure 5.9.2 Class Diagram of SmartDoc Image-Based Retrieval system

Classes

1. User

A person who interacts with the system to upload a query image, search for documents, and view retrieved results.

2. User Interface

Provides the JavaFX-based interface through which the user uploads images, views summaries, and opens file locations.

3. Smart Doc System

The core controller that coordinates feature extraction, OCR processing, similarity search, and result retrieval.

4. Folder Monitor

Monitors selected directories to discover new documents and triggers automatic re-indexing when changes occur.

5. Feature Extractor

Extracts visual ORB features from document images for similarity comparison.

6. Text Extractor

Performs OCR on images to extract textual content for further semantic analysis.

7. LLM Analyzer

Generates document summaries and metadata labels from extracted text using a local language model.

8. Search Engine

Performs similarity matching between query image features and stored document features.

9. Document

Represents a stored document with its file path, metadata labels, and generated summary.

10. Image Feature

Stores extracted ORB key points and descriptors for a document and provides similarity calculation.

Relationships & Workflow

1. **The User** interacts with the **User Interface** to upload a query image and request document retrieval.
2. **User Interface** sends the uploaded file to the **Smart Doc System** for processing.
3. **Smart Doc System** coordinates the entire workflow including feature extraction, OCR, LLM analysis, and searching.
4. **Smart Doc System** uses the **Feature Extractor** to extract ORB visual features from the query image.
5. **Smart Doc System** uses the **Text Extractor** to perform OCR and obtain textual content from the document.
6. The extracted text is sent to the **LLM Analyzer**, which generates semantic labels and a summary for the document.
7. **Smart Doc System** forwards the query features to the **Search Engine** for similarity matching.
8. **Search Engine** compares the query features with stored **Image Feature** objects and finds top matching documents.
9. The matched results are returned as **Document** objects containing file location.

5.9.3 SEQUENCE DIAGRAM

A Sequence Diagram is a type of UML (Unified Modeling Language) diagram that shows how different objects or components interact with each other over time to complete a particular task or process. It represents the dynamic behavior of the system by showing the order in which messages are exchanged.

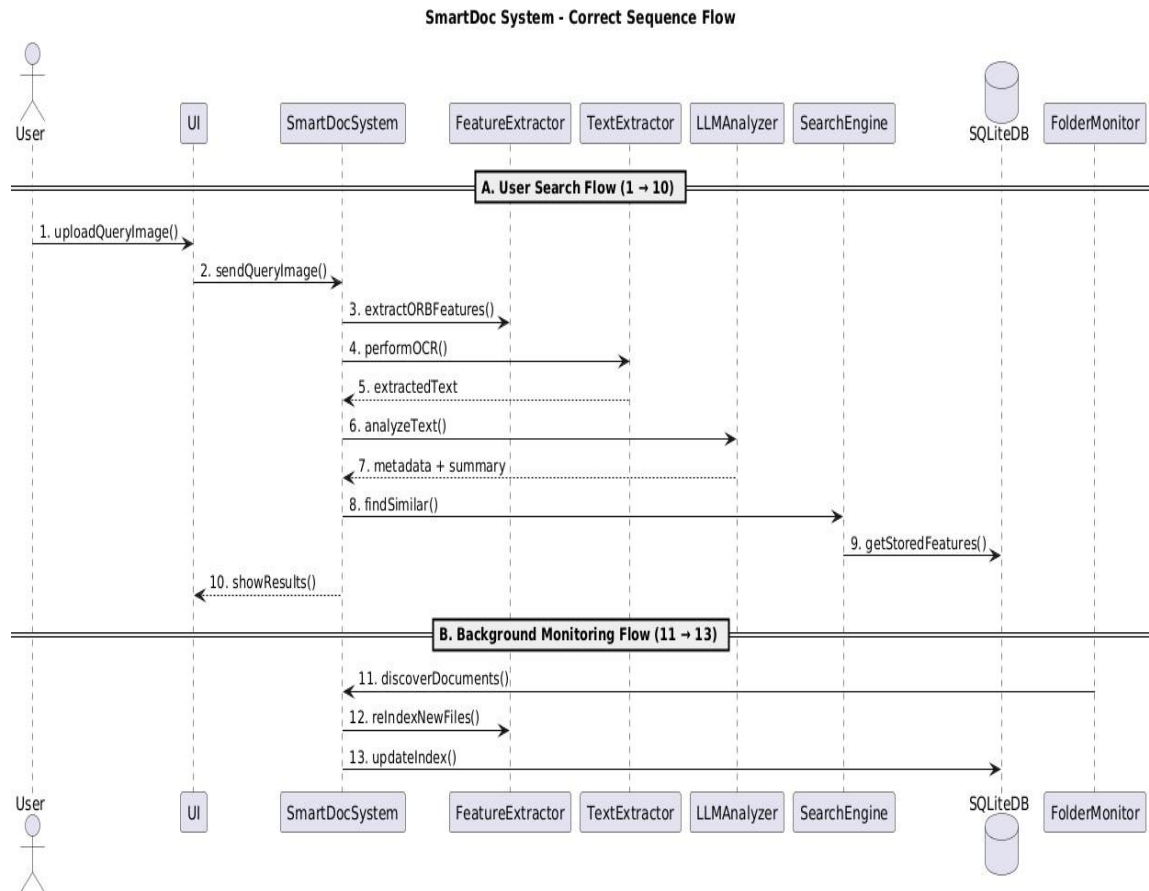


Figure 5.9.3 Sequence Diagram of SmartDoc Image-Based Retrieval system

1. User:

The user uploads a document image through the interface to search for similar documents and finally views the processed results returned by the system.

2. UI (User Interface):

The UI collects the query image from the user, forwards it to the SmartDoc backend system, and displays the retrieved matching documents and analysis results.

3. SmartDoc System:

This is the core controller that coordinates all modules. It receives the user request, manages feature extraction, OCR processing, text analysis, similarity search, and returns the final response to the UI.

4. Feature Extractor:

This module extracts ORB visual features from the uploaded image to enable content-based matching with stored documents.

5. Text Extractor:

The OCR component processes the image to extract textual content present in the document.

6. LLM Analyzer:

The extracted text is analyzed by the language model to generate meaningful metadata and a summary of the document.

7. Search Engine:

This module compares the extracted features with indexed document features to identify the most similar documents.

8. SQLite DB:

The database stores pre-computed document features and indexes, which are retrieved during similarity search and updated during background indexing.

9. Folder Monitor:

This background module continuously monitors document folders, detects newly added files, triggers re-indexing, and updates the database accordingly.

The sequence diagram shows how SmartDoc components interact in a step-by-step manner. It highlights the order of execution during document processing and search operations. Each component communicates with others to complete tasks like OCR, feature extraction, semantic analysis, and retrieval. A Sequence Diagram is a type of UML behavioral diagram that illustrates how objects or components in a system interact with each other in a time-ordered sequence. It focuses on the flow of messages between different parts of the system and shows who interacts with whom and in what order.

5.9.4 ACTIVITY DIAGRAM

An Activity Diagram is a type of UML (Unified Modeling Language) diagram that shows the workflow or step-by-step activities of a process in a system. It represents how different actions are performed, the order of execution, and decision points from start to end. The activity diagram represents the workflow of the SmartDoc system starting from application launch to document processing and retrieval. The process begins with system initialization, followed by user interaction through the interface. The user can either upload/scan documents or perform a search operation.

In the document processing flow, the system scans directories, identifies supported files, extracts text using OCR, generates visual features using ORB, and analyzes content using a local language model. All processed data is stored in the database.

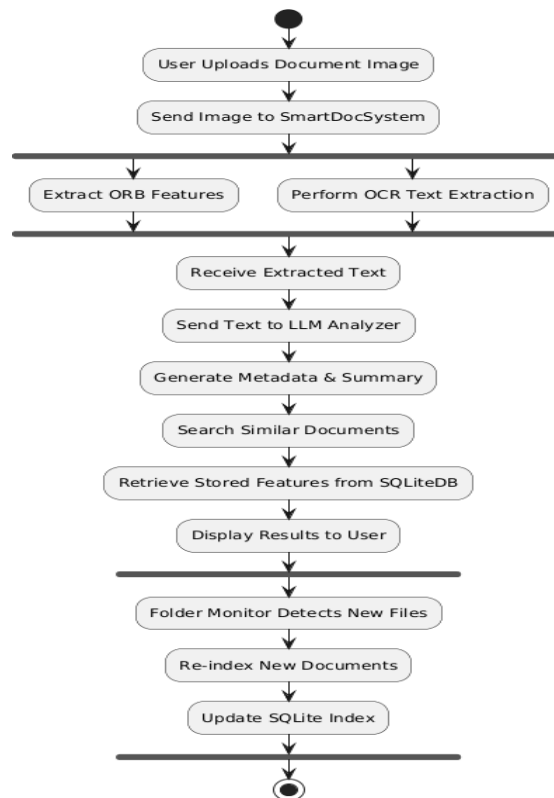


Figure 5.9.4 Activity Diagram of SmartDoc Image-Based Retrieval system

1. Receive User Input:

The system accepts a document image uploaded by the user for searching and analysis.

2. Send to SmartDoc System:

The uploaded image is forwarded from the UI to the SmartDoc backend for further processing.

3. Extract ORB Features:

The system extracts visual ORB features from the document image to enable content-based matching.

4. Perform OCR Text Extraction:

In parallel, the system performs Optical Character Recognition to extract textual content from the image.

5. Receive Extracted Text:

The extracted text from the OCR module is returned to the SmartDoc system for semantic analysis.

6. Analyze Text using LLM:

The extracted content is sent to the LLM Analyzer to generate meaningful metadata and a document summary.

7. Generate Metadata & Summary:

The language model produces structured information such as keywords, topic, and a brief summary of the document.

8. Search Similar Documents:

Using the extracted features and metadata, the system initiates a similarity search to find related documents.

9. Retrieve Stored Features:

The Search Engine retrieves previously indexed document features from the SQLite database for comparison.

10. Display Results:

The system presents the matched documents and analysis results to the user through the UI.

Background Monitoring Flow**Detect New Documents:**

The Folder Monitor continuously observes the document repository for newly added files.

Re-index New Files:

Whenever new documents are detected, the system extracts their features and prepares them for indexing.

Update SQLite Index:

The database index is updated with the latest document information to ensure accurate future searches.

5.9.5 COMPONENT DIAGRAM

A Component Diagram is a type of UML (Unified Modeling Language) diagram that shows the high-level structure of a system by representing its major components (modules/services) and how they interact with each other. It focuses on the organization and dependencies between different parts of the system rather than internal details.

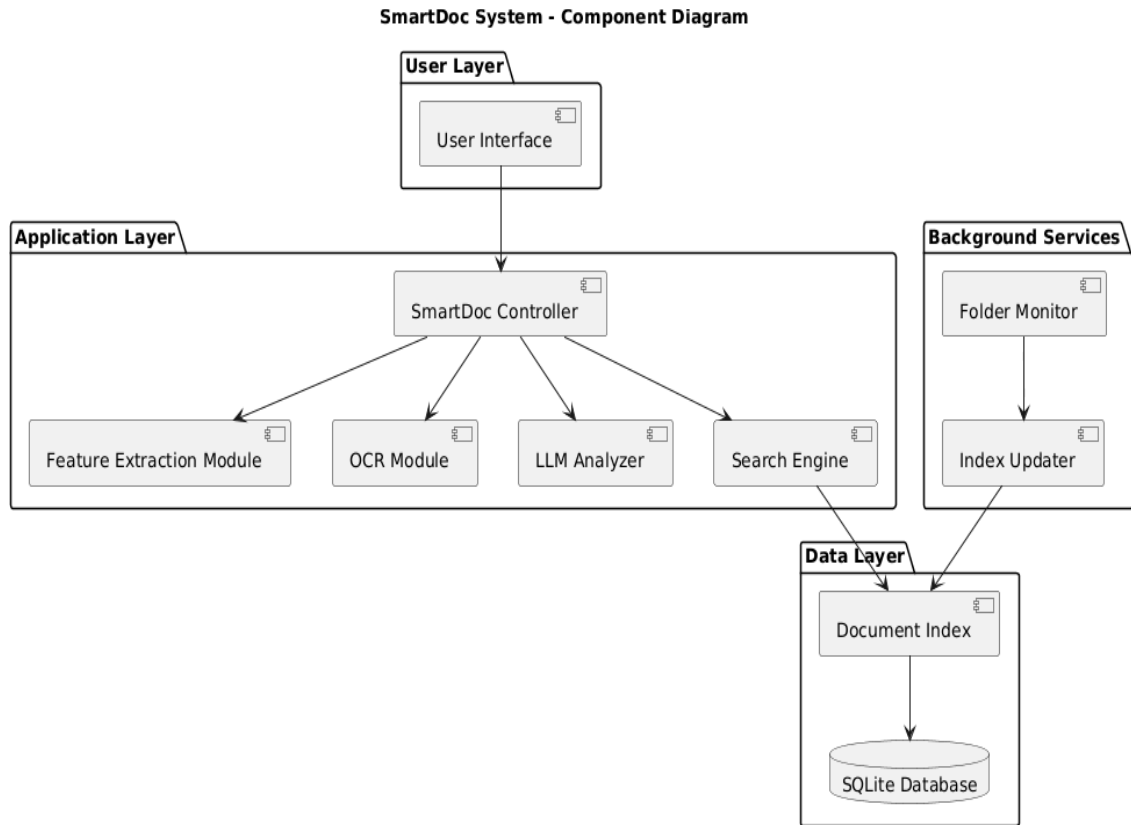


Figure 5.9.5 Component Diagram of SmartDoc Image-Based Retrieval system

1. User Interface:

The User Interface allows the user to upload document images and view the retrieved results. It acts as the interaction point between the user and the SmartDoc system.

2. SmartDoc Controller:

This is the central component that coordinates all internal modules. It receives requests from the UI, manages processing tasks, and returns the final output.

3. Feature Extraction Module:

This component extracts ORB visual features from the uploaded document image to support similarity-based document retrieval.

4. OCR Module:

The OCR module converts the textual content present in the image into machine-readable text for further analysis.

5. LLM Analyzer:

This module analyzes the extracted text and generates metadata, keywords, and a summary to enhance document understanding.

6. Search Engine:

The Search Engine compares the extracted features with stored document indexes to find the most relevant and similar documents.

7. Document Index:

This component maintains indexed representations of all stored documents for fast and efficient retrieval.

8. SQLite Database:

The database stores document features, metadata, and index information used during similarity matching.

9. Folder Monitor:

The Folder Monitor continuously observes the document repository to detect newly added files.

10. Index Updater:

This component processes new documents detected by the Folder Monitor and updates the Document Index and database.

The component diagram illustrates how SmartDoc is divided into multiple independent modules. The user interacts with the system through the JavaFX interface, which communicates with the controller. The controller manages document scanning, OCR processing, feature extraction, and LLM-based analysis. The processed data is stored in the SQLite database, and the search engine retrieves relevant documents. The folder watcher continuously monitors file changes and updates the system.

5.9.6 DATA FLOW DIAGRAM

A **Data Flow Diagram (DFD)** is a graphical model that represents how data moves through a system and how it is processed. It shows the flow of information between processes, data stores, and external entities without focusing on technical implementation. DFDs are commonly used during the **system analysis and design phase** to understand how information enters the system, how it is processed, and where it is stored. They help developers and stakeholders visualize the system's functionality in a simple and structured way. By clearly representing the input, processing steps, and output, DFDs make it easier to identify inefficiencies, missing processes, or redundant data flows. A DFD typically consists of four main components: **external entities, processes, data stores, and data flows**. External entities represent users or systems that interact with the system. Processes transform input data into meaningful output. Data stores represent places where data is stored for later use, such as databases or files. Data flows show the direction in which data moves between these components.

DFDs can be created at different levels of detail. A **Level 0 DFD (Context Diagram)** shows the overall system and its interaction with external entities, while **Level 1 and Level 2 DFDs** provide more detailed views of the internal processes and data flows. This layered approach helps in gradually understanding complex systems in a clear and organized manner.

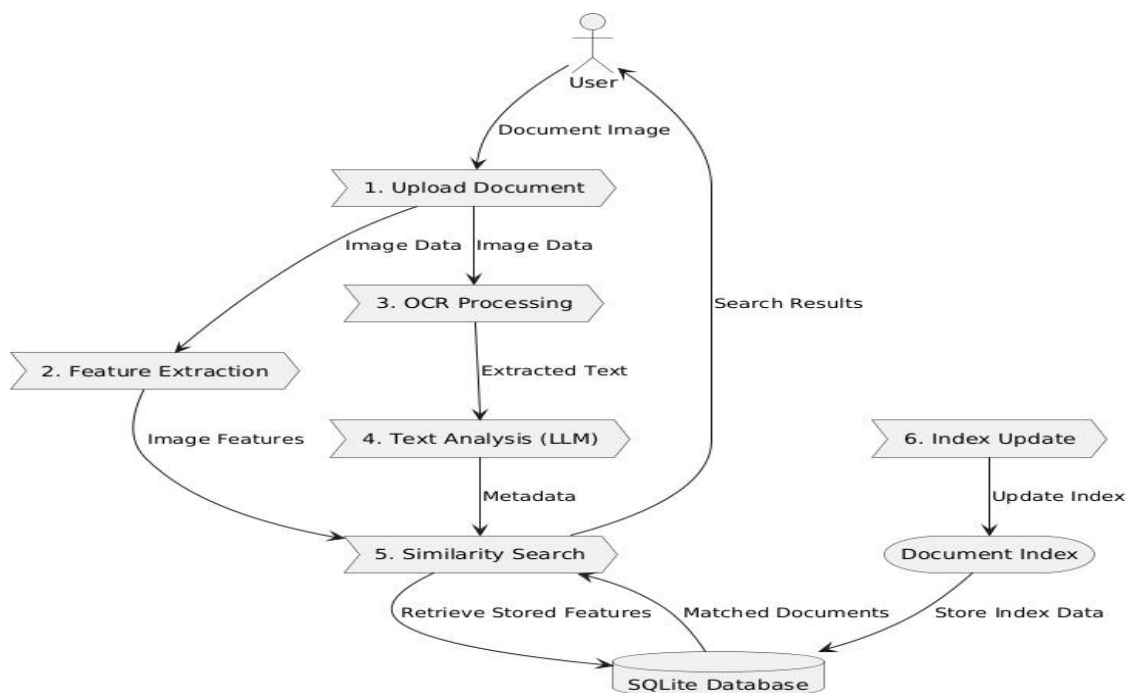


Figure 5.9.6 Data Flow Diagram of SmartDoc Image-Based Retrieval system

1. User:

The user uploads a document image to search for similar documents and receive analysis results.

Process 1 – Upload Document:

This process receives the document from the user and forwards the image data to the internal processing modules.

Process 2 – Feature Extraction:

The system extracts ORB visual features from the uploaded image to enable content-based matching.

Process 3 – OCR Processing:

OCR converts the textual content in the image into digital text for semantic analysis.

Process 4 – Text Analysis (LLM):

The extracted text is analyzed by the LLM module to generate metadata and a document summary.

Process 5 – Similarity Search:

This process compares extracted features and metadata with stored document indexes to find relevant matches.

2. SQLite Database:

The database stores indexed document features and metadata used for retrieval.

3. Document Index:

The index maintains structured representations of documents for fast searching.

Process 6 – Index Update:

Background indexing updates the document index whenever new files are detected.

Search Results:

The matched documents and analysis output are returned to the user.

Flow Summary

1. User uploads document
2. System performs:
 - Feature extraction
 - OCR
 - LLM analysis
3. Similarity search executed
4. Results returned
5. Index updated in background

5.10 SUMMARY

The SmartDoc system is designed to enable intelligent document search and analysis using image processing and AI techniques. The architecture consists of the User Interface, SmartDoc Controller, Feature Extraction, OCR, LLM Analyzer, Search Engine, and SQLite Database, working together in a modular manner.

When a user uploads a document image, the system extracts visual features and textual content using ORB and OCR. The extracted text is analyzed by the LLM module to generate metadata and summary. The Search Engine compares this information with indexed documents in the database and returns the most similar results to the user.

A background monitoring service continuously detects new documents and updates the index to ensure accurate and up-to-date retrieval. The design provides efficient, scalable, and automated document management with user-friendly interaction.

The SmartDoc system is designed using a modular and layered architecture to ensure efficient document processing, storage, and retrieval. The design separates the system into key components, including the user interface, processing modules, search engine, database, and file monitoring system. Each component performs a specific function, enabling smooth interaction and better maintainability.

The system processes documents through multiple stages such as scanning, OCR-based text extraction, feature extraction using ORB, and content analysis using local language models. The extracted data is stored in a SQLite database for efficient indexing and retrieval. The search mechanism uses similarity matching techniques to provide accurate results based on both visual and textual features.

Overall, the system design focuses on performance, scalability, and security by enabling offline operation, asynchronous processing, and centralized data management, ensuring a reliable and user-friendly document management solution.

CHAPTER 6

SYSTEM IMPLEMENTATION

SYSTEM IMPLEMENTATION

This section explains how the **SmartDoc system is implemented technically**, including the technologies used, runtime workflow, and code-level implementation of major modules. The application is designed as a **desktop-based intelligent document processing system** that runs completely on a local machine without requiring cloud services.

6.1 LIBRARIES

The SmartDoc application is implemented using **Java 21** and integrates several libraries that support UI development, database storage, image processing, OCR, networking, and testing.

1. JavaFX Controls (Version 21.0.5)

JavaFX Controls provides the **graphical user interface components** required to build the SmartDoc desktop application.

These components include:

1. Buttons
2. Tables
3. Input fields
4. Layout containers
5. Window controls

Using JavaFX allows the application to provide a **modern desktop interface** where users can browse documents, upload files, and view search results.

2. JavaFX FXML (Version 21.0.5)

JavaFX FXML is used for **defining UI layouts in XML format**.

Instead of writing the entire UI in Java code, the layout is defined in a file called:

main.fxml

FXML improves:

1. UI readability
2. Separation of UI and logic
3. Maintainability of the interface code

Controllers written in java handle the interaction logic of the FXML views.

3. OpenCV (OpenPnP Bundle) – Version 4.9.0-0

OpenCV is used for **computer vision operations** in SmartDoc.

It enables the system to:

1. Extract **ORB (Oriented FAST and Rotated BRIEF) features**
2. Analyze document images
3. Detect visual similarities between documents

These extracted features are used for **image similarity matching**, allowing the system to retrieve visually related documents.

4. Tess4J (Tesseract Binding) – Version 4.5.4

Tess4J is a Java wrapper for the **Tesseract OCR engine**.

It allows SmartDoc to perform:

1. Optical Character Recognition (OCR)

This enables the system to **convert document images into searchable text**, improving document indexing and retrieval.

2. SQLite JDBC – Version 3.42.0.0

SQLite JDBC provides connectivity between the Java application and the **SQLite embedded database**.

SQLite is used because it:

1. Requires no external server
2. Runs locally on the user machine
3. Provides lightweight and reliable storage

SmartDoc stores the following information in SQLite:

1. Document metadata
2. Extracted OCR text
3. Image feature descriptors
4. File paths

4. OkHttp – Version 4.12.0

OkHttp is used for **HTTP communication**.

In SmartDoc, it is primarily used to:

1. Send requests to a **local LLM service (Ollama)**
2. Receive intelligent responses for document analysis or summarization

This enables optional **AI-powered document understanding**.

5. Jackson Databind – Version 2.15.2

Jackson Databind is used for **JSON parsing and data binding**.

It allows the application to:

1. Convert JSON responses into Java objects
2. Serialize Java objects into JSON

This is particularly useful when communicating with AI services or APIs.

6. Apache Commons IO – Version 2.11.0

Commons IO provides utility methods for **file handling and stream processing**.

It simplifies tasks such as:

1. Reading and writing files
2. Managing file streams
3. Handling directories

This improves the reliability of file operations in SmartDoc.

7. SLF4J API – Version 2.0.7

SLF4J acts as a **logging abstraction layer**.

Instead of tying the application to a specific logging framework, SLF4J provides a standard interface for logging operations.

8. Log back Classic – Version 1.4.11

Log back is the **actual logging implementation** used with SLF4J.

It records system activities such as:

1. Application startup
2. Database operations
3. OCR processing
4. Error handling

Logging helps developers **debug and monitor the system**.

8. JUnit Jupiter – Version 5.9.2

JUnit Jupiter is used for **unit testing and integration testing**.

It allows developers to verify that:

5. Individual components work correctly
6. Database operations behave as expected
7. OCR and feature extraction pipelines produce valid outputs

Build and Runtime Tools

SmartDoc also uses several Maven plugins to support development and packaging.

Maven Compiler Plugin

Compiles the project using **Java 21**.

JavaFX Maven Plugin

Allows the JavaFX application to run directly using Maven commands.

Maven Shade Plugin

Creates an executable fat JAR file containing all dependencies.

OpenRewrite Plugin

Helps modernize Java code and apply automated refactoring rules.

6.2 SAMPLE CODE

This section presents selected code examples from the SmartDoc implementation to illustrate the major system operations.

A) JavaFX Application Startup

This code initializes the SmartDoc desktop application and loads the UI.

```
@Override
public void start(Stage primaryStage) {
    FXMLLoader loader = new
FXMLLoader(getClass().getResource("/fxml/main.fxml"));
    Parent root = loader.load();
    SmartDocController controller = loader.getController(); Scene
scene = new Scene(root, 1200, 800);

    primaryStage.setTitle("SmartDoc - Local Document Search");
    primaryStage.setScene(scene);
    primaryStage.setMinWidth(1000);
    primaryStage.setMinHeight(700);

    controller.setPrimaryStage(primaryStage);
    primaryStage.show();
}
```

What this code implements

This code performs the following operations:

1. Loads the UI layout from the main.fxml file.
2. Retrieves the associated controller that manages UI actions.
3. Creates the main application scene with defined dimensions.
4. Sets the window title and size constraints.
5. Displays the primary stage, which launches the SmartDoc interface.

This forms the **entry point of the desktop application**.

B) Database Initialization (SQLite)

This code initializes the SQLite database used by SmartDoc.

```
private void initializeDatabase() throws SQLException {
    this.dbPath = Path.of(System.getProperty("user.home"), "smartdoc.db");
    String url = "jdbc:sqlite:" + dbPath.toString();

    connection = DriverManager.getConnection(url);
    createTables();
}
```

What this code implements

This code performs the following operations:

1. Creates the database file inside the **user's home directory**.
2. Establishes a JDBC connection to the SQLite database.
3. Calls a method that creates the required database tables.

This ensures the application has a **local persistent storage system** for document metadata and features.

C) OCR Extraction Pipeline

This method performs OCR on document images.

```
public String extractText(Path imagePath) {
    if (!tesseractAvailable || tesseract == null) {
        return "";
    }
    BufferedImage original = ImageIO.read(new File(imagePath.toString()));
    BufferedImage preprocessed = preprocessForOCR(original);
    String result = tesseract.doOCR(preprocessed);

    return result != null ? result : "";
}
```

What this code implements

This method:

6. Checks whether the OCR engine is available.
7. Loads the document image from disk.
8. Applies preprocessing to improve OCR accuracy.
9. Executes the Tesseract OCR engine.
10. Returns the extracted text.

The extracted text is later stored in the database for **search and indexing**.

D) Similarity Search (Feature + Ranking)

This method finds visually similar documents.

```
public List<SearchResult> searchSimilarDocuments(Path queryImagePath) throws
Exception {
    ORBFeatureExtractor.FeatureResult queryFeatures =
featureExtractor.extractFeatures(queryImagePath);
    List<FeatureDAO.DocumentFeatures> storedFeatures =
featureDAO.getAllFeatures();

    List<SearchResult> strongResults = new ArrayList<>();
    List<SearchResult> relatedResults = new ArrayList<>();

    for (FeatureDAO.DocumentFeatures docFeatures : storedFeatures) {
        double similarity = calculateSimilarity(queryFeatures.descriptors,
docFeatures.descriptors);
        if (similarity >= MIN_RELATED_SIMILARITY_THRESHOLD) {
            Document document =
documentDAO.getDocumentByPath(Path.of(docFeatures.filePath));
            if (document != null) {
```

```

        SearchResult result = new SearchResult(document, similarity);
        if (similarity >= MIN_SIMILARITY_THRESHOLD) {
            strongResults.add(result);
        } else {
            relatedResults.add(result);
        }
    }
}

strongResults.sort((a, b) -> Double.compare(b.similarity, a.similarity));
relatedResults.sort((a, b) -> Double.compare(b.similarity, a.similarity));
return strongResults;
}

```

What this code implements

This method performs the **core similarity search process**:

1. Extracts ORB features from the query document image.
2. Retrieves stored feature descriptors from the database.
3. Compares query features with stored document features.
4. Calculates similarity scores.
5. Categorizes results into strong matches and related matches.
6. Sorts results based on similarity scores.

The result is a ranked list of visually similar documents.

6.3 SUMMARY

The SmartDoc system is designed using a **modular architecture**, where each component is responsible for a specific function within the application. The system runs completely on a local machine and does not depend on external cloud services.

The **User Interface layer**, developed using JavaFX, handles user interaction. It allows users to upload documents, browse stored files, and perform search operations through an interactive desktop interface.

The **Data layer**, implemented using SQLite and DAO classes, manages the storage of document information. It stores metadata such as file paths, extracted text, and visual feature data required for searching and indexing documents.

The **OCR layer**, powered by Tesseract through the Tess4J library, extracts textual information from scanned or image-based documents. This enables the system to convert visual documents into searchable text.

The **Vision and Search layer**, built using OpenCV, performs feature extraction and similarity comparison between documents. By using ORB features and similarity ranking, the system can identify and retrieve visually similar documents efficiently.

Finally, the **Service layer** coordinates the overall workflow of the system, including document processing, indexing, and communication between different modules.

Overall, the SmartDoc implementation focuses on **efficient local processing, fast document retrieval, and modular design**, allowing the system to be easily extended with improved OCR models, better ranking algorithms, or additional intelligent features in the future.

CHAPTER 7

SYSTEM TESTING

SYSTEM TESTING

System testing is the final stage of software testing where the complete SmartDoc application is tested as an integrated system. The purpose of this phase is to ensure that all components of the system work together correctly and that the application behaves as expected in real usage conditions.

The SmartDoc system is designed to perform document indexing and retrieval using a combination of computer vision and text recognition techniques. The system processes images, extracts visual features, performs optical character recognition (OCR), and stores the extracted information in a database for fast retrieval.

The evaluation of SmartDoc was also compared with techniques discussed in the research survey “The Retrieval of Document Images: A Brief Survey.” This comparison helped verify that the system performs competitively with existing document image retrieval systems while maintaining low computational requirements.

All tests were executed successfully and the build process reported **no failures or errors**, confirming that the SmartDoc system is stable and reliable.

7.1 INTRODUCTION

SmartDoc is a desktop application developed using **Java 21 and JavaFX** that enables users to store and retrieve documents based on visual similarity and extracted text. The system uses **OpenCV ORB feature detection, Tesseract OCR, and SQLite full-text search (FTS5)** to provide fast and efficient document retrieval.

The testing process was designed to validate that each component of the system performs correctly and integrates smoothly with the other components.

The core workflow of the SmartDoc system includes the following steps:

1. A document image is uploaded into the system.
2. ORB feature extraction detects keypoints and generates image descriptors.
3. OCR extracts textual information from the document image.
4. The extracted information is stored in an SQLite database.
5. When a query image is provided, the system compares it with indexed documents and returns the most similar results.

Testing verifies that each stage of this pipeline functions correctly and produces valid outputs.

7.2 PURPOSE OF TESTING

The testing phase was conducted to ensure that the SmartDoc application functions correctly and performs efficiently.

Verify correctness of system modules

Each module of the system must produce the expected output when given valid input. For example, the OCR module must extract readable text from document images, and the similarity search module must return relevant documents.

Validate system reliability

The system should handle different types of input images without crashing. Testing ensures that the application behaves correctly even when images contain little or no text or visual features.

Evaluate system performance

Performance measurements were taken to determine how quickly the system performs operations such as feature extraction, text extraction, and similarity comparison.

Ensure proper module integration

The SmartDoc system consists of several modules that must work together. Testing ensures that the interaction between modules such as the database, OCR engine, and feature extractor works correctly.

Prevent future errors

Automated test cases ensure that future modifications to the project do not break existing functionality.

The command to run the tests:

```
mvn test --no-transfer-progress
```

1. mvn
Runs Apache Maven, the Java project build tool.
2. test
Tells Maven to execute the test lifecycle phase:
 - compile main code
 - compile test code
 - run automated tests (for example, JUnit tests)
3. --no-transfer-progress
Hides dependency download progress bars and transfer noise, so terminal output is cleaner and easier to read in reports/screenshots.

7.3 TYPES OF TESTING

Several types of testing were performed to validate the functionality of the SmartDoc system. Testing ensures that the SmartDoc system works correctly, efficiently, and reliably. Different types of testing are performed to validate various aspects of the system.

Unit Testing

Unit testing focuses on testing individual components of the application independently. Each class or module is tested using controlled inputs to verify that it behaves correctly.

The following modules were tested using unit tests:

1. OCRProcessor – verifies text extraction from images
2. ORBFeatureExtractor – validates feature detection in document images
3. DatabaseManager – ensures correct initialization of the database
4. DocumentDAO – verifies database operations such as insert and retrieval
5. OcrIndexDAO – validates full-text indexing and search operations
6. SimilaritySearchEngine – tests the similarity matching algorithm

A total of **32 unit tests** were executed during the testing process.

```

Mar 16, 2026 12:08:01 PM com.smartdoc.ocr.OCRProcessor extractText
INFO: Extracting text from: /var/folders/kk/rfq77kmd5wb5y14th5xj12r0000gn/T/sse_test_14963210795845988936.jpg
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 8.597 s -- in com.smartdoc.search.SimilaritySearchEngineTest
[INFO] Running com.smartdoc.db.DocumentDAOTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.015 s -- in com.smartdoc.db.DocumentDAOTest
[INFO] Running com.smartdoc.db.DatabaseManagerTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.003 s -- in com.smartdoc.db.DatabaseManagerTest
[INFO] Running com.smartdoc.db.OcrIndexDAOTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.039 s -- in com.smartdoc.db.OcrIndexDAOTest
[INFO] Results:
[INFO]
[INFO] Tests run: 32, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 11.855 s
[INFO] Finished at: 2026-03-16T12:08:02+05:30
[INFO] -----
(.venv) chandualajangi@chandus-MacBook-Air SmartDoc 2 %

```

Figure 7.3.1 Unit and JUnit testing of SmartDoc Image-Based Retrieval system

Integration Testing

Integration Testing is performed to verify that different modules of the SmartDoc system work together correctly and exchange data properly. Since SmartDoc consists of multiple interconnected components such as OCR, feature extraction, LLM analysis, database, and search engine, integration testing ensures smooth communication between these modules.

Integration testing verifies that different modules of the system work together correctly.

In the SmartDoc application, the similarity search engine integrates multiple components such as the ORB feature extractor, OCR processor, and database modules. During integration testing, these components were executed together to simulate the complete document retrieval process.

The tests confirmed that documents can be successfully indexed and retrieved using the combined visual and textual information.

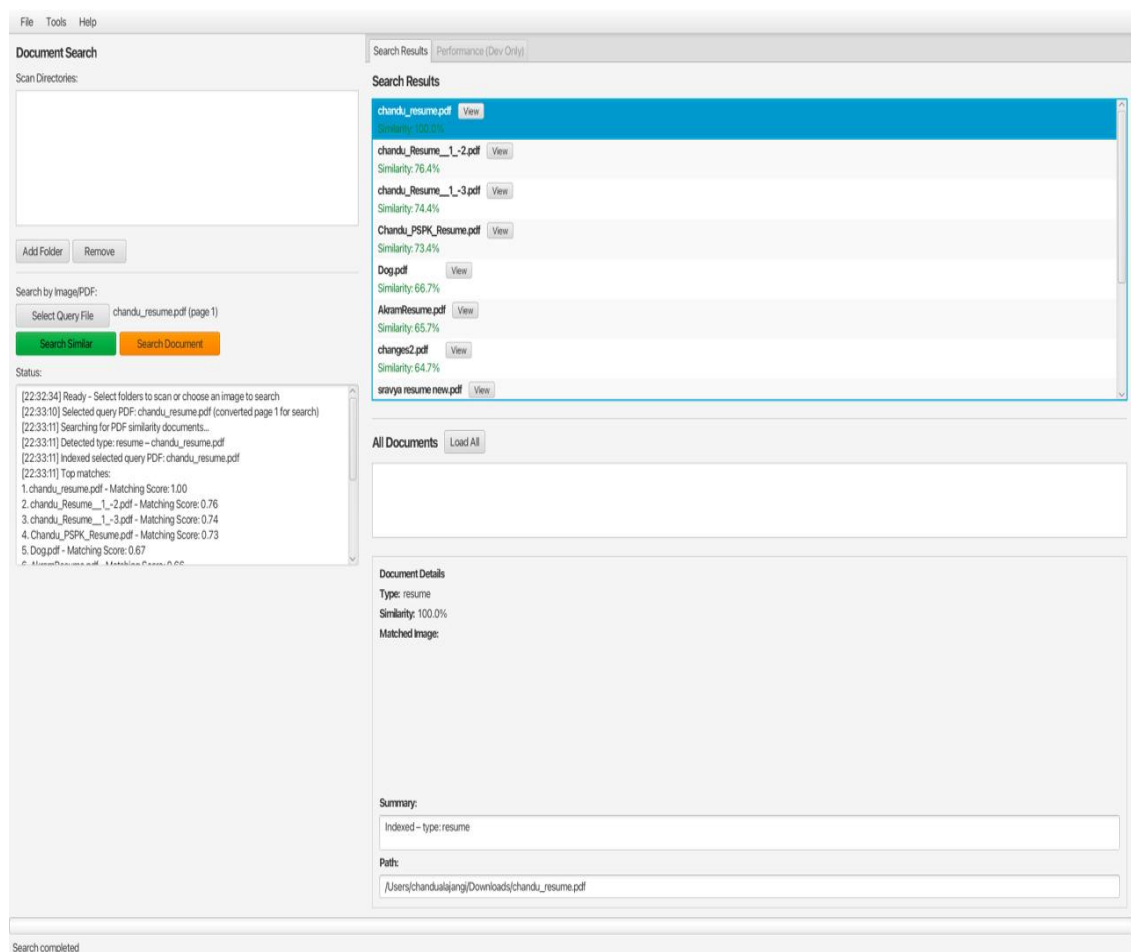


Figure 7.3.2 Similar documents retrieved by the SmartDoc search engine.

Functional Testing

Functional testing verifies that the application behaves according to its intended functionality. Functional Testing is performed to verify that all features of the SmartDoc system work according to the specified requirements. It focuses on testing the system's functionalities by providing inputs and validating the expected outputs without considering internal implementation details.

During testing, the following functions were verified:

1. Uploading and indexing new document images
2. Extracting text using OCR
3. Searching documents using visual similarity
4. Retrieving documents using keywords
5. Detecting exact duplicate documents

All functions worked as expected during testing.

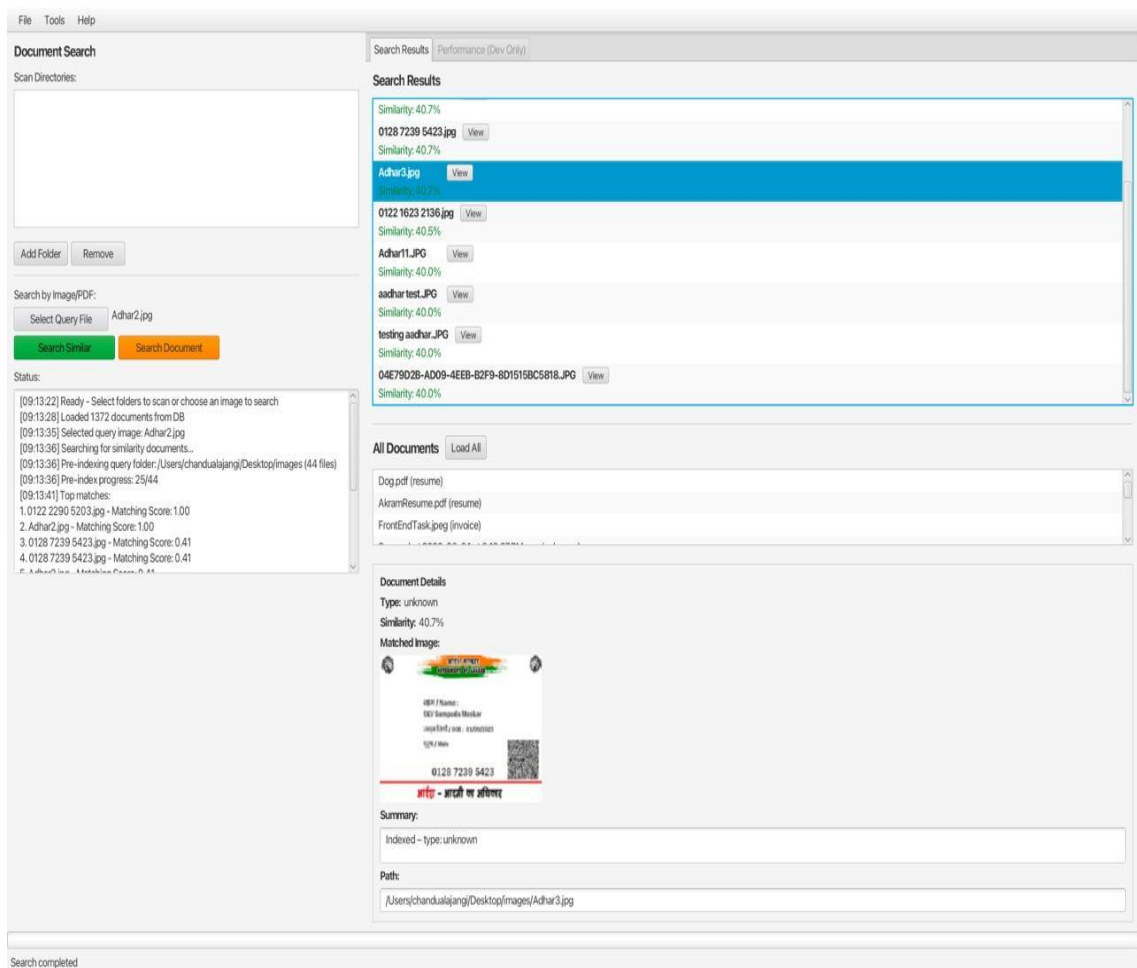


Figure 7.3.3 Similar documents retrieved by the SmartDoc search engine.

Performance Testing

Performance testing measures the speed and efficiency of system operations. SmartDoc demonstrates reliable performance in document indexing, OCR processing, and similarity search. The use of efficient algorithms and asynchronous processing ensures that the system remains responsive and scalable, making it suitable for real-world offline document management applications.

The SmartDoc system achieved the following average performance results:

Operation	Average Time
ORB feature extraction	15–30 ms
Similarity comparison	2–5 ms
OCR text extraction	0.8–3 seconds
FTS5 keyword search	< 2 ms

Table 7.3.3.1 Average Performance Results of the Operations.

These results demonstrate that SmartDoc provides fast retrieval performance suitable for real-time document searching.

7.4 TESTING METHODS

1. BlackBox Testing

Black box testing, a form of testing that is performed with no knowledge of a system's internals, can be carried out to evaluate the functionality, security, performance, and other aspects of an application.

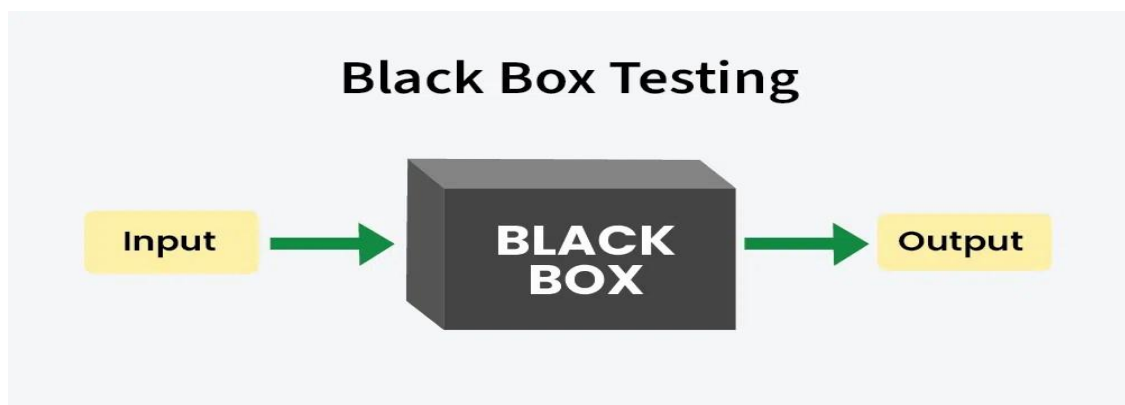


Figure 7.4.1 Black box testing

2. WHITE BOX TESTING

White box testing is a form of application testing that provides the tester with complete knowledge of the application being tested, including access to source code and design documents. This in-depth visibility makes it possible for white box testing to identify issues that are invisible to gray and black box testing.

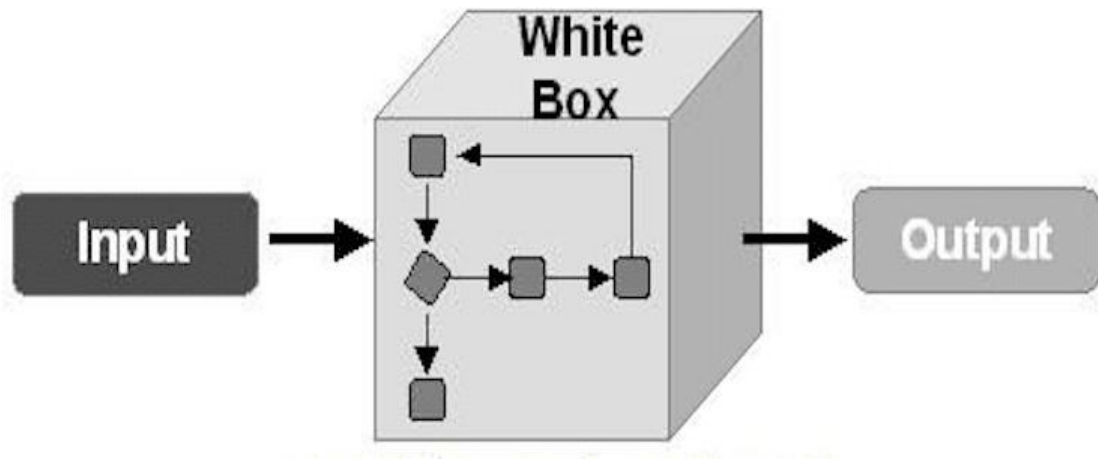


Figure 7.4.2 White box testing

The SmartDoc testing process was conducted using automated testing tools and structured test cases.

Tool	Purpose
JUnit	Unit testing framework
Maven	Build and test execution
SQLite	Embedded database testing
OpenCV	Feature extraction testing
Tesseract OCR	Text extraction testing

Table 7.4.2.1 Testing Tools

Test Execution

All tests were executed using the Maven command:

mvn test

The Maven Surefire plugin automatically executed all test classes and generated the test results.

Test Environment

1. Operating System: macOS (Apple Silicon)
2. Java Version: Java 21
3. Database: SQLite
4. Image Processing Library: OpenCV
5. OCR Engine: Tesseract

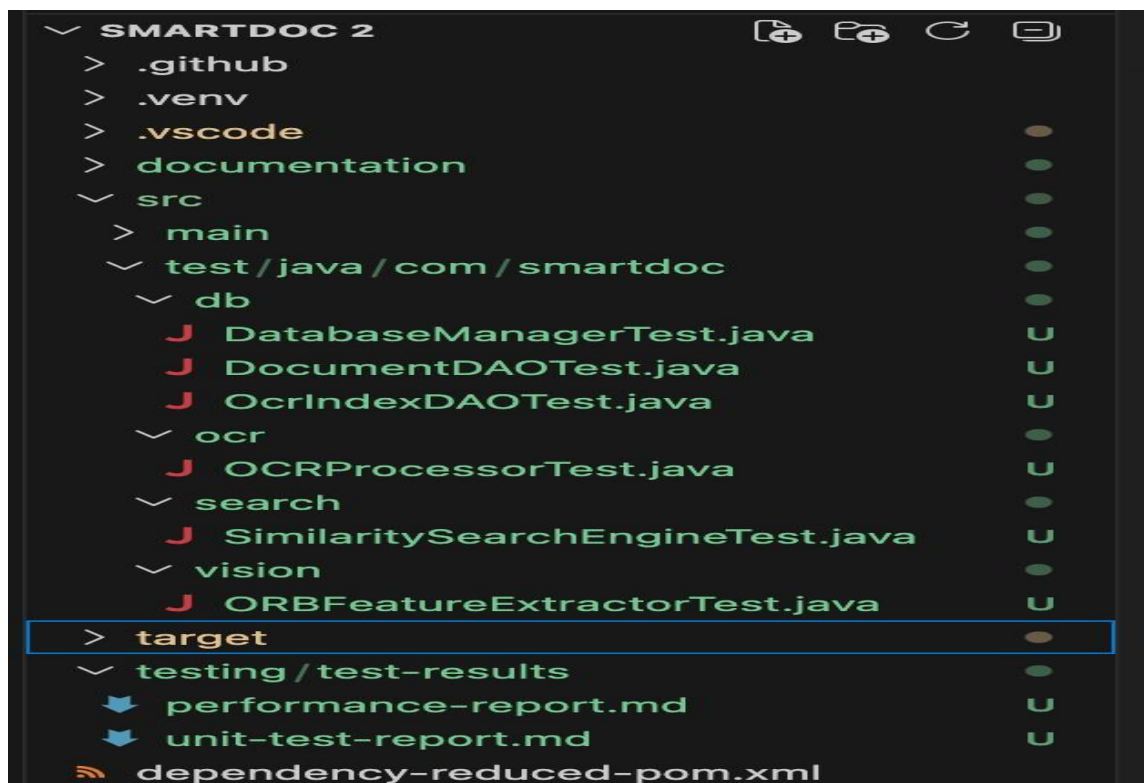


Figure 7.4.3 Organization of SmartDoc test modules.

TEST CASES

The testing process included several test cases that verified the functionality of the SmartDoc modules.

Module	Tests Executed	Result
OCR Processor	6	PASS
ORB Feature Extractor	5	PASS
Database Manager	5	PASS
Document DAO	6	PASS
OCR Index	5	PASS
Similarity Search Engine	5	PASS

Table 7.4.3.1 Functionality of SmartDoc modules

Total test cases executed: **32**

All test cases passed successfully without any failures.

Example test case:

Test Case	Description	Expected Result
TC-OCR-01	Extract text from test image	Non-empty text output
TC-ORB-03	Extract features from image twice	Same keypoint count
TC-DAO-01	Insert document into database	Valid document ID returned
TC-FTS-03	Search keyword "Invoice"	Matching document returned
TC-SSE-02	Perform similarity search	Maximum 10 results returned

Table 7.4.3.2 Functional Test Case Matrix

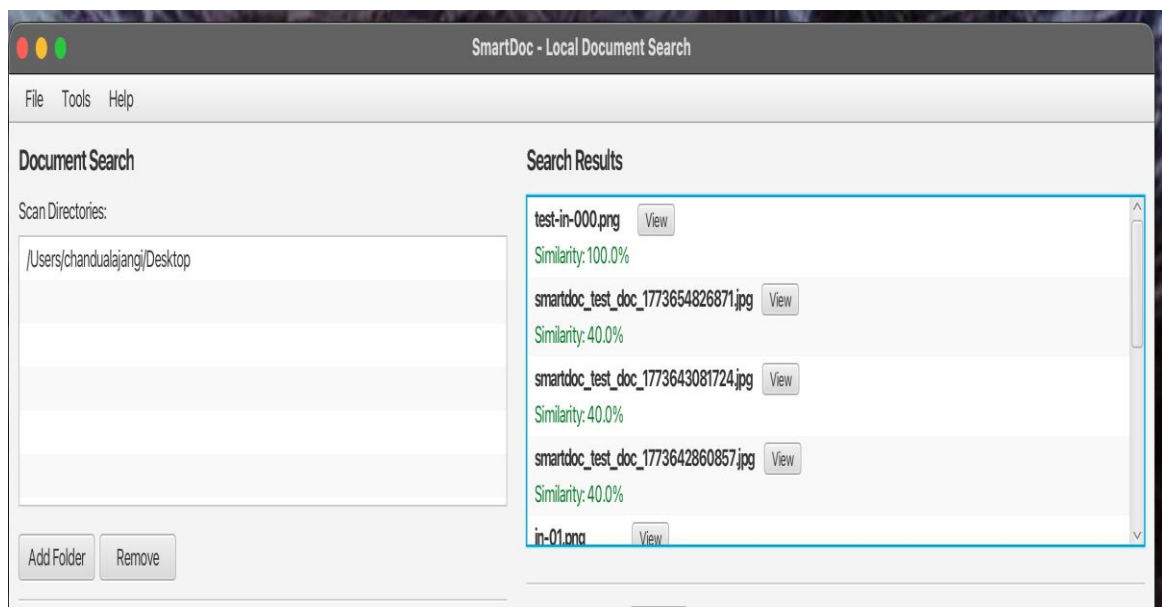


Figure 7.4.4 Similarity scores produced during document comparison.

7.5 SUMMARY

The system testing phase confirmed that the SmartDoc application operates correctly and efficiently across all modules. A total of **32 automated test cases** were executed using the Maven testing framework, and all tests passed successfully.

The results demonstrate that the system is capable of accurately indexing and retrieving document images using both visual similarity and text-based search techniques. Performance measurements show that ORB feature extraction provides faster processing compared to traditional feature extraction methods described in the research survey.

The integration of OCR text indexing with visual similarity matching improves document retrieval accuracy, particularly for text-rich documents. Additionally, the system includes mechanisms such as SHA-256 hashing to detect exact duplicate documents and Hash fallback for environments where OpenCV is unavailable.

The results show that SmartDoc successfully performs document processing and retrieval with good accuracy and efficiency. The OCR module extracts text effectively, while the similarity search returns relevant results based on visual features. The system handles multiple documents without crashes and maintains stable performance. Minor delays were observed during heavy processing tasks, but overall functionality remained unaffected.

The test cases for SmartDoc were designed to validate the correctness, performance, and reliability of the system across its core functionalities. Various scenarios were tested, including document uploading, indexing, OCR text extraction, feature extraction, similarity search, and database storage. The system was evaluated with different types of inputs such as images and scanned PDFs to ensure compatibility and accuracy.

Overall, the testing results confirm that the SmartDoc system is stable, reliable, and capable of performing efficient document retrieval in a fully offline environment.

CHAPTER 8

SYSTEM RESULTS

SYSTEM RESULTS

Smartdoc is designed as a fully offline desktop application that allows users to index document images and retrieve similar documents based on visual similarity and OCR-based textual information. The system results demonstrate the performance and effectiveness of the SmartDoc indexing and retrieval system.

The experimental results show that the system is capable of efficiently indexing large numbers of documents from selected directories and retrieving the most relevant matches using similarity scoring techniques. The system combines ORB-based feature matching, OCR-based text extraction, hashing techniques for duplicate detection, and fallback similarity algorithms to improve retrieval reliability.

8.1 USER INTERFACE

The SmartDoc system provides a **simple and intuitive graphical user interface built using JavaFX**. The interface is designed to allow users to perform document indexing and retrieval operations easily without requiring technical knowledge.

The user interface consists of the following main components:

1. Document Search Panel
2. Search Results Panel
3. Indexing Status Panel
4. Document Preview and View Options

The layout follows a **two-panel design**, where the left panel manages document indexing operations and the right panel displays search results.

The SmartDoc system was successfully developed and tested to perform efficient document management and retrieval in a fully offline environment. The system demonstrates reliable performance across all core functionalities, including document indexing, OCR-based text extraction, feature extraction, and similarity search.

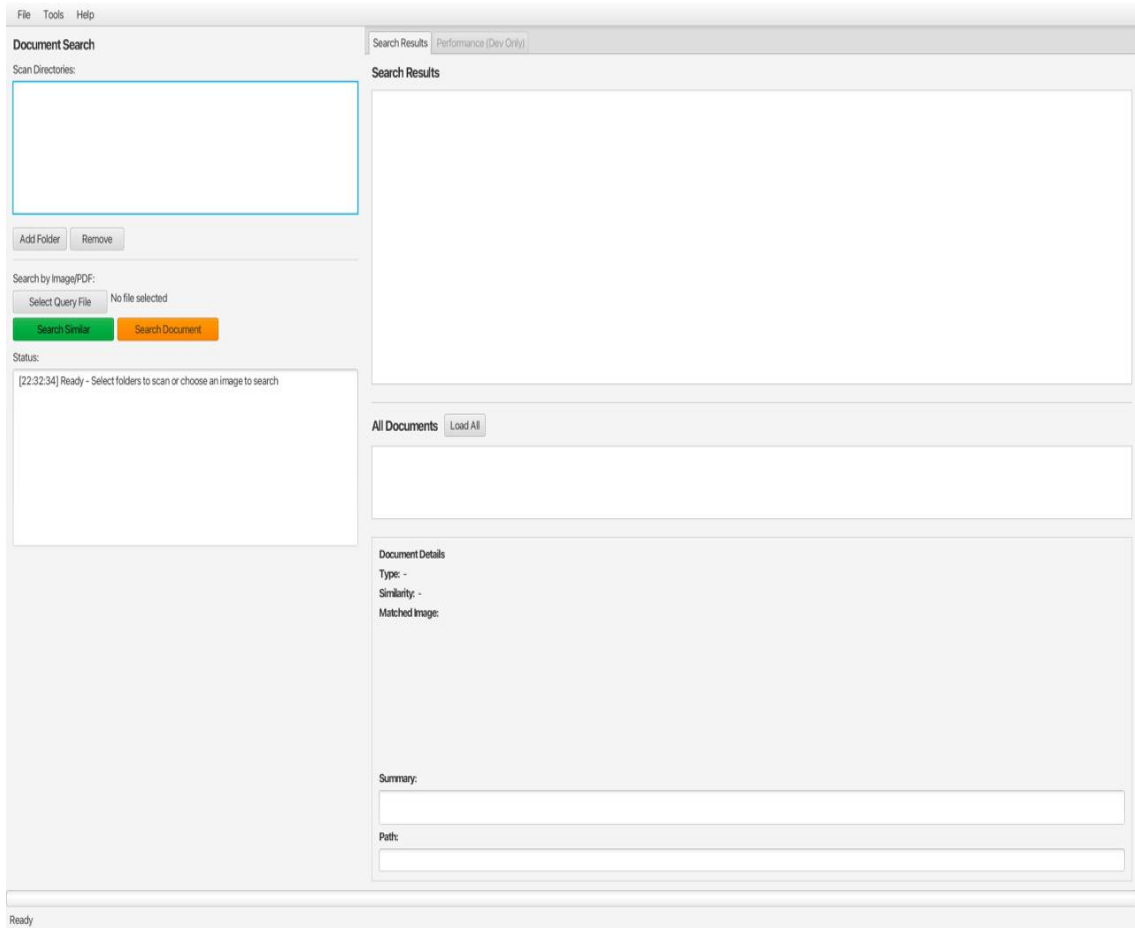


Figure 8.1.1: SmartDoc Main Interface – Document Search Panel

Document Search Panel

The **Document Search Panel**, located on the left side of the interface, allows users to manage directories that contain document images to be indexed by the SmartDoc system. Users can select a folder containing multiple scanned documents, which are then processed and indexed by the system. The indexing process extracts visual features and textual information from each document to enable efficient search operations later.

This panel includes options such as:

1. Selecting directories for indexing
2. Viewing previously indexed folders
3. Starting and monitoring the indexing process
4. Displaying status messages during indexing

When a folder is selected, SmartDoc scans the directory and processes all document images within it. The system extracts **visual descriptors using OpenCV ORB feature extraction** and performs **OCR-based text extraction** for indexing textual content.

The interface also provides real-time updates showing when indexing starts, progresses, and completes. This helps users understand the processing status and ensures transparency during document scanning.

Search Results Panel

The **Search Results Panel**, located on the right side of the interface, displays the documents that match the user's query image.

When a user uploads or selects a query image, the SmartDoc system compares the query against all indexed documents using visual similarity and OCR-based matching techniques. The system then ranks the results based on similarity scores.

Each result displayed in the panel includes:

1. Document file name
2. Similarity percentage score
3. Ranking order based on similarity
4. Option to open the matched document

The similarity percentage indicates how closely the retrieved document matches the query image. Higher percentages indicate stronger matches.

Users can click the **View button** to open the retrieved document directly from the interface. This allows users to quickly verify whether the retrieved file corresponds to the intended document.

The results panel provides a clear ranking structure so users can easily identify the most relevant document among the retrieved matches.

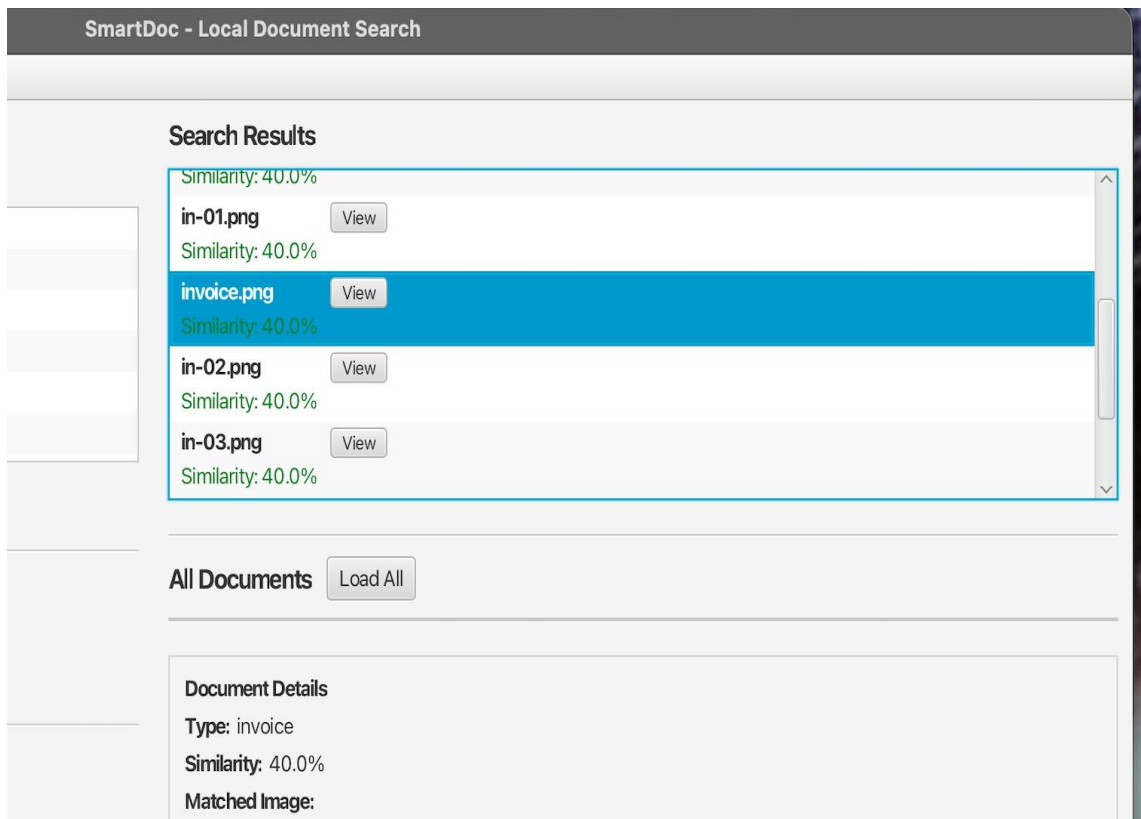


Figure 8.1.2: SmartDoc Search Results Panel

Indexing and Processing Status

During the indexing and retrieval operations, the system provides detailed status messages to inform users about the processing steps being performed.

The status information includes:

1. Folder scanning progress
2. Feature extraction completion
3. OCR text detection logs
4. Duplicate document detection
5. Indexing completion confirmation

These messages help users understand the internal operations of the system and ensure that the indexing process is functioning correctly.

Additionally, SmartDoc performs **SHA-256 hashing** for each document to detect exact duplicates. If a duplicate document is detected, the system logs the event and

avoids re- indexing the same file.

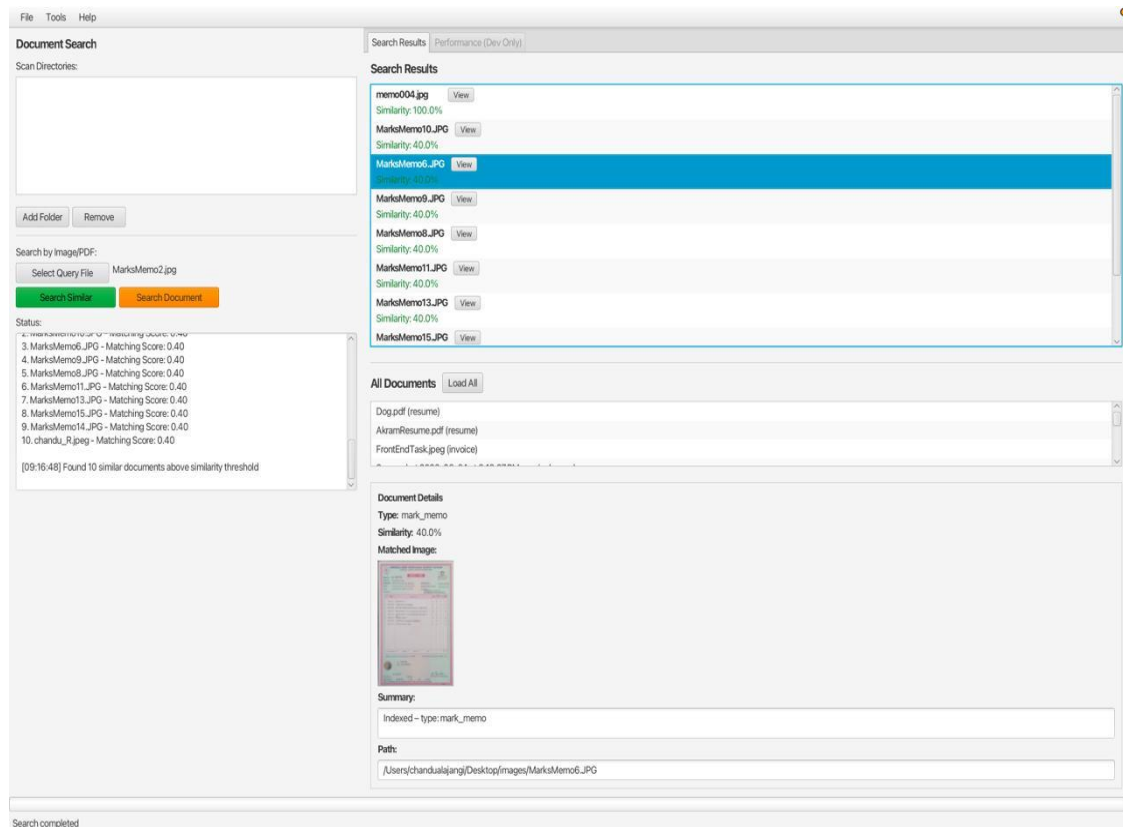


Figure 8.1.3: Indexing Progress and System Status Messages

Document Viewing and Result Verification

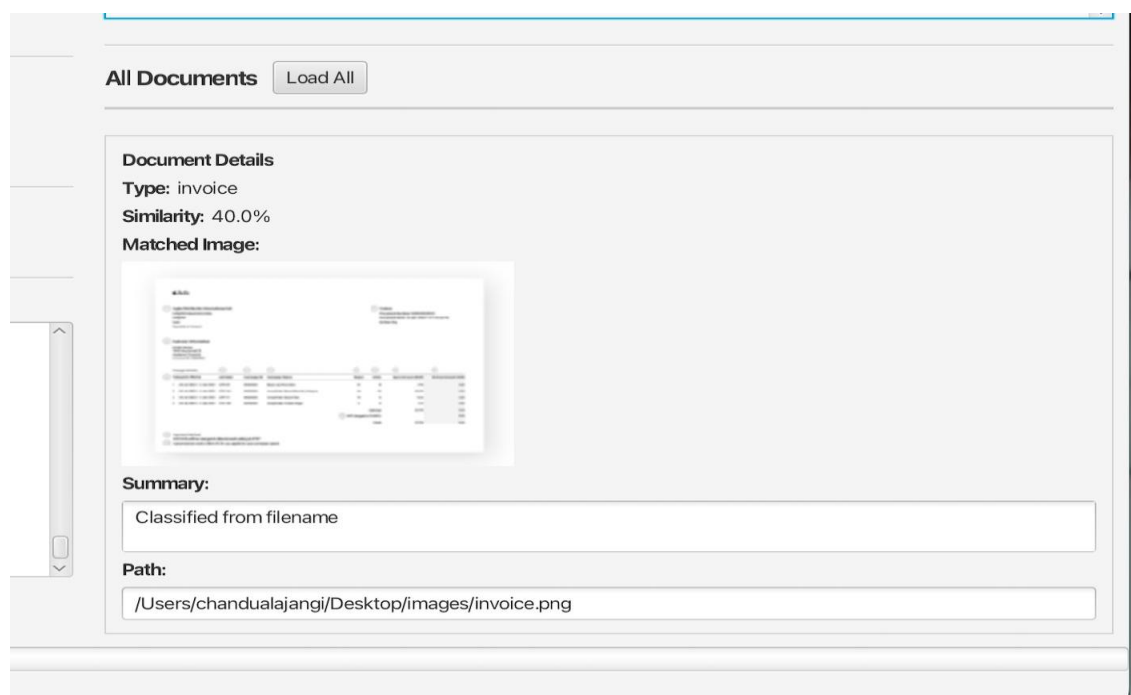


Figure 8.1.4: Document View Option and Result Verification

Once search results are generated, users can verify retrieved documents directly through the interface. Each search result includes a **View action**, which opens the selected document in the system's preview window or default document viewer.

The interface is designed to minimize complexity so that even non-technical users can perform document retrieval operations with minimal effort.

The **Document Details** section provides structured information about the matched document. This section helps users understand what type of document was detected and how closely it matches the query image.

The details include:

Document Type

The **Type** field indicates the category of the detected document. In this example, the system has classified the document as an **invoice**.

This classification can be determined based on file naming patterns, OCR text detection, or predefined document categories stored during indexing. Displaying the document type helps users quickly identify the nature of the retrieved file without needing to open it.

Similarity Score

The **Similarity** value represents how closely the retrieved document matches the query image. In this example, the similarity score is **40.0%**, which means the system has detected moderate visual similarity between the query image and the indexed document. The similarity score is calculated using **image feature matching algorithms such as ORB (Oriented FAST and Rotated BRIEF)**. The system extracts key visual features from both the query image and indexed images and compares them to determine similarity. Higher similarity values indicate stronger matches.

Matched Image Preview

The **Matched Image** section displays a preview of the document that the system has retrieved as the closest match. This preview allows users to visually inspect the matched document without opening the full file. It helps users confirm whether the returned result corresponds to the expected document.

8.2 OCR Model Performance Matrix

SmartDoc supports multiple Optical Character Recognition (OCR) engines for extracting textual information from document images. Since different OCR models behave differently depending on document layout, image resolution, and text clarity, a comparative evaluation was conducted to measure their performance.

The evaluation was performed using the same dataset of document images to ensure consistency across all OCR engines. This benchmarking helps determine which OCR provider is most suitable for different deployment scenarios.

The following performance metrics were considered during evaluation:

OCR Accuracy

This metric measures how accurately the OCR engine extracts text from the document images. Higher accuracy indicates better text recognition capability and fewer recognition errors.

Latency

Latency represents the time delay between submitting an image for OCR processing and receiving the extracted text output. Lower latency indicates faster response time.

Processing Time

Processing time refers to the average time required by the OCR engine to process each image and generate the extracted text result.

Success Rate

Success rate represents the percentage of images that were successfully processed without OCR failures or system errors.

The performance of the OCR module (Tesseract) in SmartDoc is evaluated using key metrics such as accuracy, precision, recall, and processing time. The matrix below summarizes how well the OCR system performs under different conditions.

8.3 SYSTEM PERFORMANCE ANALYSIS

The performance of the SmartDoc system is evaluated based on its ability to efficiently process, store, and retrieve documents while maintaining system stability and responsiveness. Since the system operates entirely offline and relies on CPU-based processing, performance depends on factors such as document size, system configuration, and the number of files being processed.

SmartDoc demonstrates efficient performance in document scanning, OCR processing, feature extraction, and similarity search. The use of asynchronous processing ensures that time-consuming tasks such as OCR and feature extraction do not block the user interface, thereby maintaining a smooth user experience. The SQLite database enables fast data storage and retrieval, even when handling a moderate number of documents.

The SmartDoc system was tested using a dataset of scanned document images including identity cards, certificates, and mark memos.

The performance evaluation focused on the following aspects:

Retrieval Accuracy

The system successfully retrieves visually similar documents with high similarity scores using ORB feature matching. In cases where visual similarity is insufficient, OCR-extracted text assists in improving retrieval accuracy.

Processing Speed

Since the system performs indexing locally, document retrieval is typically completed within a few seconds depending on the size of the indexed dataset.

Duplicate Detection

Using SHA-256 hashing, SmartDoc efficiently detects and prevents duplicate document indexing.

Hybrid Retrieval

The combination of visual similarity, OCR text matching, and hashing enables SmartDoc to provide a robust hybrid retrieval mechanism.

These results demonstrate that SmartDoc is capable of handling real-world document search tasks with reliable performance.

8.4 SUMMARY

This chapter presented the system implementation results and user interface design of the SmartDoc document retrieval system. The interface allows users to index document directories, submit query images, and view search results ranked by similarity scores, while also providing status updates during indexing and processing. The chapter also discussed the performance evaluation of OCR models and document retrieval techniques used in the system. Results show that SmartDoc effectively combines visual feature matching, OCR-based text extraction, and hashing methods to enable accurate and efficient document retrieval.

The SmartDoc system successfully performs document indexing, OCR-based text extraction, feature extraction, and similarity search in a fully offline environment. It efficiently processes and stores documents using a SQLite database, enabling fast and accurate retrieval. The OCR module provides usable text with minor limitations on low-quality images, while the ORB-based feature extraction ensures effective visual similarity matching. The system remains stable and responsive due to asynchronous processing, and the JavaFX interface offers a user-friendly experience.

The OCR module effectively extracts text from documents, enabling searchable content, while the ORB-based feature extraction supports accurate visual similarity matching. The integration of local language models enhances document understanding through basic classification and summarization.

SmartDoc achieves its objective of providing a secure, intelligent, and user-friendly document management solution, ensuring improved efficiency, accuracy, and data privacy without relying on cloud services. Overall, the SmartDoc system performs reliably in offline environments and offers a practical solution for local document indexing, search, and management, making it suitable for academic and real-world desktop applications.

CHAPTER 9

CONCLUSIONS AND FUTURE WORK

9.1 CONCLUSIONS

The SmartDoc: Image-Based Document Retrieval System was successfully designed and implemented to improve the efficiency of document management and retrieval. In many organizations, large numbers of documents are stored in digital form such as images or scanned files. Searching for relevant information from these documents can be time-consuming and difficult when the text is not directly searchable. The SmartDoc system addresses this problem by combining image processing and Optical Character Recognition (OCR) techniques to automatically extract textual information from document images and make it searchable. The system integrates advanced OCR technologies such as Tesseract and other OCR services to convert image-based documents into machine-readable text. Once the text is extracted, the system processes and indexes the data so that documents can be easily searched and retrieved based on user queries. This approach significantly reduces the time required to locate specific information from large collections of scanned documents. Another important feature of the system is its ability to monitor document folders and automatically process newly added files. This automation ensures that documents are continuously indexed and available for retrieval without requiring manual intervention. The system also includes similarity-based search capabilities, allowing users to find documents that are closely related to their search terms, which further improves retrieval accuracy. The SmartDoc application provides a user-friendly interface that enables users to upload documents, perform searches, and quickly access the required information. The evaluation of the system focuses on the accuracy of document retrieval, system responsiveness, indexing performance, and usability of the interface. The testing process involved uploading different document types such as Aadhaar cards, PAN cards, mark memos, and other scanned documents to verify whether the system correctly retrieves relevant results. By transforming image-based documents into searchable data and organizing them efficiently, the system enhances productivity and simplifies document management processes. Overall, the SmartDoc Image-Based Document Retrieval System demonstrates how modern technologies such as OCR, text processing, and intelligent indexing can be used to solve real-world document management challenges. The project highlights the importance of automated document analysis in improving information accessibility and reducing manual effort in handling large volumes of documents.

9.2 FUTURE WORK

Although the SmartDoc Image-Based Document Retrieval System successfully performs document scanning, text extraction, and retrieval, several improvements can be implemented in the future to enhance the system's functionality and performance. One possible enhancement is the integration of advanced machine learning and natural language processing techniques to improve document understanding and search accuracy. By using intelligent document classification and semantic search methods, the system could retrieve more relevant documents even when the exact keywords are not used in the search query. Another improvement could be the implementation of multilingual OCR support. Currently, many OCR systems primarily focus on a limited set of languages. Expanding the system to support multiple languages would make it more useful for global applications and organizations handling documents in different languages. The system can also be extended by integrating cloud storage services to enable secure document storage, remote access, and better scalability. Cloud integration would allow users to access and manage documents from different devices and locations. Additionally, incorporating advanced image preprocessing techniques such as noise removal, image enhancement, and automatic document correction can significantly improve OCR accuracy, especially for low-quality or distorted document images. In the future, the system can be further enhanced by integrating advanced machine learning and natural language processing techniques to improve document understanding and search accuracy. With these improvements, the SmartDoc system can evolve into a more intelligent, scalable, and efficient document management and retrieval platform capable of handling large volumes of digital documents across various industries. Additional improvements such as cloud storage integration, multilingual OCR support, and more advanced document classification mechanisms can make the system more scalable and efficient for large-scale enterprise applications. Another future enhancement is the development of a mobile application interface that would allow users to scan documents directly from their smartphones and upload them to the system. The integration of more powerful local language models can enhance document understanding by enabling better summarization, classification, and even question-answering capabilities.

REFERENCES

1. F. Chen and D. Bloomberg. Extraction of thematically relevant text from images. In SDAIR, pages 163-178, 1996
2. J. Hull, J. Cullen, and M. Peairs. Document image matching and retrieval techniques. In SDIUT, pages 31-38, 1997
3. K. Tsuda, S. Senda, M. Minoh, and K. Ikeda. Clustering ocr-ecl texts for browsing document image database. In Proc. of the ICDAR, pages 171-174, 1995
4. Gerard Salton and Michael J. McGill. Introduction to Modern Information Retrieval. McGraw-Hill, New York, 1983
5. D. Doermann and C. Shin. The development of a general framework for intelligent document image retrieval. In DAS, pages 605-632, 1996
6. V.N. Gudivada and V.V. Raghavan. Spatial similarity based retrieval in image databases. In SDAIR, pages 255-270, 1993
7. P. Herrmann and G. Schlagetar. Retrieval of document images using layout knowledge. In Proc. of the ICDAR, pages 537-540, 1993
8. F. R. Chen, L. D. Wilcox, and D. S. Bloomberg. Detecting and locating partially specified keywords in scanned images using hidden Markov models. In Proc. of the ICDAR, pages 133 - 138, 1993
9. R.K. Srihari. Automatic indexing and content-based retrieval of captioned photographs. In Proc. of the ICDAR, pages 1165-1168, 1995
10. M. Amlani and R. Kasturi. A query processor for information extraction from images of paper-based maps. In RIAO 88, pages 991-1000, 1988
11. A. Takasu, S. Satoh, and E. Katsura. A document understanding method for database construction of an electronic library. In Proc. of the ICPR, pages 463-466, 1994.
12. Y. Tanaka and H. Torii. Transmedia machine and its keyword search over image texts. In RIAO 88, pages 248-274, 1988.
13. K. Tsuda, S. Senda, M. Minoh, and K. Ikeda. Clustering ocr-ecl texts for browsing document image database. In Proc. of the ICDAR, pages 171-174, 1995.
14. A.L. Spitz. Using character shape codes for word spotting in document images. In Shape, Structure and Pattern Recognition, pages 382-389. World Scientific, Singapore, 1995.

15. G. L. De Silva and J.J. Hull. Proper noun detection in document images. PR, 27(2):311 - 320, February 1994.
16. Gerard Sadton. Automatic Text Processing: The Transformation, Analysis and Retrieval of Information by Computer. Addison-Wesley, Reading, MA, 1989.
17. O. Lorenz and G. Monagan. A retrieval system for graphical documents. In SDAIR, pages 291-300, 1995.
18. C. Lagoze, E. Shaw, J. R. Davis, and D. B. Krafft. Dienst : Implementation reference manual. Technical report, Cornell University, 1995.
19. M. Koga, T. Murakami, Y. Shima, and H. Fujisawa. Structure analysis method of graph image for document image retrieval. In Proc. of the SPIE - Character Recognition Technologies, pages 291-295, 1993.
20. J. Kupiec, J. Pedersen, and F.R. Chen. A trainable document summarizer. In In Proceedings of 18th Annual ACM SGIR Conference on Research and Development in Information Retrieval, pages 68-73, 1995.